MECHANICAL ENGINEERING
SENIOR DESIGN CAPSTONE
FINAL REPORT

Pinball Machine Learning

A Design Project Final Report submitted in partial

fulfillment of the requirements for the degree of

BACHELOR OF SCIENCE

in

MECHANICAL ENGINEERING

and

ELECTRICAL ENGINEERING

To the Department of Mechanical and Materials Engineering,
College of Engineering and Applied Science at the University of Cincinnati.

Submitted: 4/28/2026

2025-2026 Academic Year

Mason Pierce, Electrical Engineering Team Lead, EECE5001

Noah Sergent, Electrical Engineering, EECE5001

Brenden Shelley, Mechanical Engineering, MECH5051

Dr. Jacob Cress, PE

Course Instructor

Dr. Zachariah Fuchs

Project Sponsor

Dr. Zachariah Fuchs.

Faculty and Industry Advisors

Executive Summary / Abstract

To accurately track the position of a pinball moving down a non-linear track, a large number of sensors are required. To reduce the extra material costs, manufacturing costs, and points of failure imposed by these sensors, this project proposes the creation of an MLM (machine learning model) trained to predictively determine the position of a ball along a track using a vastly reduced number of sensors. This data will be reflected by LED strips that line the length of the track and light up to show the position of the ball as it moves.

Project Description [ABET 7]

Modern pinball machines increasingly incorporate ramp-mounted RGB LED effects to enhance visual feedback and player immersion. However, existing implementations typically rely on discrete sensor triggers, preprogrammed animations, or simple linear assumptions rather than true real-time ball tracking. Achieving accurate ramp-mounted ball-tracking LED (RBL) systems using traditional approaches requires additional hardware such as cameras, large numbers of infrared sensors, and expanded GPIO resources, which can significantly increase system cost, wiring complexity, and potential points of failure. As a result, precise and adaptable RBL systems remain uncommon in commercial and hobbyist pinball machines.

This project addresses these limitations by developing a ball-tracking LED system that uses a machine learning model (MLM) to predict the real-time position of a pinball using data from only one to two beam-break sensors. By inferring ball location from minimal sensor input, the system dramatically reduces hardware requirements while maintaining high positional accuracy. The predicted ball position is transmitted to an LED controller module which drives ramp-mounted WS2812B LED strips to visually track the ball's motion.

The system consists of three primary modules: a data acquisition (DAQ) that collects sensor data, a Host PC that hosts the machine learning model, and an LED controller module responsible for rendering the predicted ball position on the ramp. The system's success criteria include an end-to-end (DAQ to LED controller) latency of less than 20ms, ensuring the visual response is imperceptible to players, and a positional accuracy within 2cm of the actual ball location.

Key design considerations include ease of integration, cost reduction, and player experience. Each module is designed to operate as a standalone component to enable seamless integration into existing pinball machines. The DAQ module requires only 2-3 ramp-mounted sensors and a TTL serial connection, the Host PC requires execution of a provided Python script, and the LED controller requires power, a serial connection to the LED strip, and a TTL serial link to the Host PC. Meeting the defined latency and accuracy requirements ensures responsive and intuitive visual feedback during gameplay, satisfying stakeholder expectations for both gameplay quality and system maintainability.

Design Constraints Identification [ABET 2, (4)]

Client and/or Stakeholder Mandated Constraints:

The client has requested that this system operate as an independent unit that can be added onto a functional pinball machine. The system should be able to handle data acquisition, compute, and light control without interfering with the primary game of pinball that is occurring.

Ethical and Professional Responsibilities:

There are no significant ethical concerns with this project.

Health and Safety Constraints and Considerations:

When fabricating certain portions of this project, there will be concerns about safety. When soldering is occurring, proper safety glasses will be worn, and a fume extractor will be used. The completed project will not pose risks to safety.

Cultural Diversity and Accessibility:

Our project will use lights to convey information. The team will ensure that information is not color coded to allow for color blind accessibility.

Economic Constraints and Considerations:

Our system should be designed in a way that the cost of parts is kept to a minimum. Since pinball machines are already complex, expensive systems, it is important to choose parts which do not have extra features that will not be utilized in the project.

Environmental Constraints and Considerations:

Fabrication of this project will require the use of soldering and 3d printing. The team will only perform these activities in spaces with proper ventilation and safety equipment, taking care that hazardous materials are properly managed and disposed of.

Sustainability Considerations:

The team will take care to minimize waste when developing this project. This includes optimizing 3d printed designs to use material more efficiently. The project will not use significant power or material, so the sustainability risk is minimal.

Legal and Regulatory Considerations:

There are no legal or regulatory considerations with this project.

Security Considerations:

There are no security considerations for the project as it does not deal with sensitive information. Additionally, there is no dangerous uncontrolled motion that could occur.

Manufacturability Considerations:

Our project should be easily and cheaply manufacturable as a way to add on to existing and future pinball machines. The module will not directly interact with other forms of processing, so it should be able to be developed on its own.

Engineering Standards and Codes Applied to the Project [ABET 2] [ETAC] Program Criteria / ETAC

For this project, many practices defined by IEEE Standard 1012 (System and Software Verification and Validation) have been utilized to guide the development and evaluation of the system. IEEE 1012 defines a structured framework for verifying a system is built correctly and validating that the system has been built to meet stakeholder needs. This standard is applicable to this project because the system integrates hardware modules, real-time communication, and a machine learning model whose output directly affects user-perceived behavior during gameplay. A requirements traceability matrix mapping each system requirement to its corresponding test procedure and classification is provided in Table 6: Requirements Traceability Matrix.

In accordance with IEEE 1012, verification tests were performed at the module level to confirm that each subsystem functions correctly in isolation. The Signal Capture Test Procedure verified the DAQ module by comparing IR sensor break durations measured with an oscilloscope against the values reported by the DAQ firmware over serial, with a pass criterion of ±500µs agreement. The LED Functionality Test Procedure verified the LED Controller module by confirming that the Host PC software could command both full-strip and individual LED activation on the WS2812B LED strip. Successful completion of these two verification tests also served to implicitly verify the serial communication between modules, as correct test results require that the framed UART protocol is functioning properly in both the transmit and receive directions.

Validation tests focused on confirming that the fully integrated system met the two primary stakeholder requirements. The System Latency Test Procedure validated end-to-end latency by measuring the time elapsed from sensor trigger through machine learning prediction to LED command output, with a pass criterion of less than 20ms. The LED Accuracy Test Procedure validated positional accuracy by comparing the location of the activated LED to the position of the pinball using frame-by-frame review of 120FPS video, with a pass criterion of the activated LED remaining within 2cm of the ball at all times during a run.

Design Considerations

Some design considerations include:

- Constructing a test track with known and controllable variables to test the machine learning model on.
- An apparatus that continually feeds the ball onto the track for collecting training data that is adjustable for different size tracks.
- All tracks are designed to be 3D printed with little to no supports for quicker production and smoother finishes
- The final code is constructed in such a way that allows for easy integration into any system.

Design Process, Alternatives, and Basis for Selection [ABET 2]

Software Methodology

The software architecture for the Ball-Tracking LED (BAL) system was developed using a top-down design methodology. The system was first separated into major components based on the required behavior.

1. Data acquisition from ramp-mounted sensors
2. Communication between DAQ, Host PC, and LED Controller
3. Processing of sensor data
4. Prediction of ball trajectory
5. LED activation timing
6. User monitoring, debug capabilities, and system control through a graphical interface.

Each of these components were integrated as a separate module or group of modules to keep a clear separation of responsibilities and to simplify debugging. The software design process involved iterative prototyping and unit testing of modules.

Software Design Alternatives

LED Animation

Two possible strategies were considered for directing the LED Controller from the Host PC. The first approach transmits LED updates only when state changes occur. The second approach maintains a full LED state buffer and periodically sends this buffer to the LED Controller.

Table 1 LED Design Alternatives

Design Alternative	Advantages	Disadvantages
Event-driven	Lower communication overhead; fewer redundant transmissions; simpler for small updates	Harder to maintain full LED state consistency; more difficult to coordinate complex animations; becomes tightly coupled with communication logic.
Periodic refresh (selected)	Clear separation between animation logic and transmission logic; deterministic LED updates; supports many animation types; More error-resistant	Higher communication overhead, requires maintaining of a full LED state buffer on Host PC and LED Controller

The periodic refresh method was selected because it provides a cleaner separation between LED control and the processing of DAQ data. By maintaining an internal buffer representing the state of every LED on the strip and periodically sending this buffer to the LED controller, the software can support multiple animation types without modification of the data processing or communication components. As a result, the software architecture is more modular and extensible, making it easier to add new animation behaviors in the future. Additionally, periodic refresh ensures that the LED Controller regularly receives the full intended state of the LED strip, improving robustness in the event of missed or corrupted transmissions.

UART Message Framing

In the serial communication implementation, two main framing options were considered: raw data streaming and a custom framed protocol. Raw data streaming would involve a fixed number of bytes transmitted to/from the MCUs and Host PC, with the receiver interpreting data based solely on byte position. The custom framed protocol involves delimiters and header fields being sent alongside the payload to mark the start, end, and type of message.

Table 2 UART Design Alternatives

Design Alternative	Advantages	Disadvantages
Raw data streaming	Simple implementation; minimal overhead; fewer bytes transmitted per message	Fragile; difficult to debug; harder to support multiple message types
Structured framed protocol (selected)	Clear message boundaries using framing bytes; easier parsing; more robust to incomplete messages or desync; more extensible	More difficult to implement; more bytes per message; requires sender and receiver to strictly follow protocol

A custom structured framed protocol was selected as the serial communication method between the modules. Although raw data streaming offers a simpler implementation and lower transmission overhead, it relies on strict byte alignment between the sender and receiver and provides no protection against synchronization errors and would only support one type of message between communicating module pair. In contrast, the framed protocol includes explicit message boundaries and metadata that allow the receiver to reliably identify the beginning and end of each message and interpret the payload based on the provided header fields. This improves robustness when handling incomplete transmissions or communication desynchronization and simplifies debugging during development and testing.

Hardware Design Alternatives:

MCU Architecture

In the system architecture design, the approaches considered for handling data acquisition and LED control were the use of a single microcontroller to perform all tasks or using separate microcontrollers for each subsystem.

Table 3 MCU Design Alternatives

Design Alternative	Advantages	Disadvantages
Single MCU architecture	Simpler hardware design; communication only needed between two devices	Increased firmware complexity; competing timing requirements between sensor acquisition and LED updates; reduced modularity

Multiple MCU architecture	Clear separation of responsibilities; simpler firmware; improved timing reliability	Additional hardware required; requires communication protocol between three devices

Machine Learning Model Design Alternatives:

Three possible machine learning model types were considered. They consisted of simple linear regression, multiple linear regression, and K nearest neighbor. Simple linear regression would consist of only using the sensor break time as an input for the model. Multiple linear regression would consist of inputting total run time, and sensor break time into the model. K nearest neighbor model would consist of comparing measured input values to collected data, and fitting it to the nearest neighbors. Advantages and disadvantages were considered for each model type and are outlined in Table 4.

Table 4 MLM Type Considerations

Model Type	Advantages	Disadvantages
Simple Linear Regression	Very quick to run Versatility Easier integration with LED control	Most simple option, could experience decreased accuracy compared to other options
Multiple Linear Regression	Moderately quick run time More accurate	Less versatile More susceptible to differences in total run time Larger number of inputs requires more training data
K Nearest Neighbor	Highest theoretical accuracy with adequate amount of training data	Slower run time could result in delays Requires significantly more training data Could suffer with edge cases

Design Overview [ABET 1]

Because of the nature of this project, it involves two main design components: the physical track design and the software design. The documentation below outlines the key structures of each component of the project.

Track Design Overview:

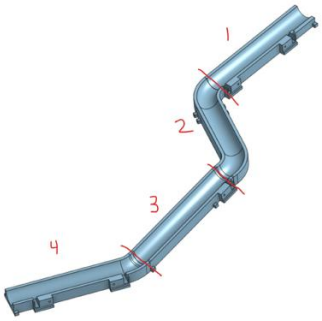


Figure 1 Final Track

The final track is split using dove tail connections to allow for ease of printing and transportation.

Dovetail:

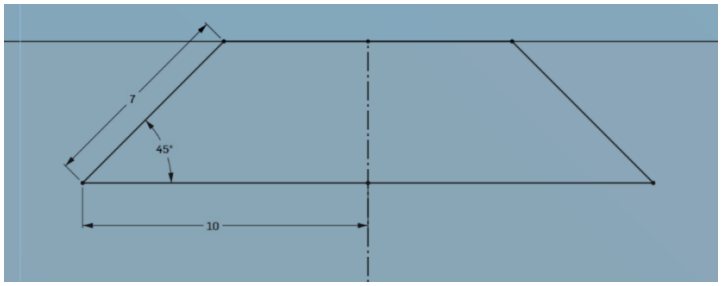


Figure 2 Dovetail Dimensions

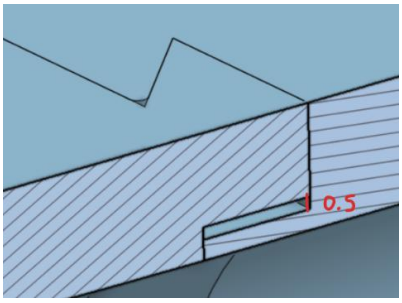


Figure 3 Dovetail Thickness

It was found that using the same dimensioning for the male and female ends of the dovetail connections resulted in a snug fit that holds the tracks firmly together (Figure 2). Having a gap of 0.5 mm between the top and bottom of the dovetail connection resulted in the track sitting flush with one another (Figure 3).

Sensor mount dimensioning:

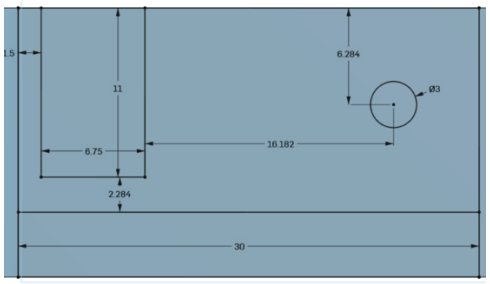


Figure 4 Sensor Mount Dimensions

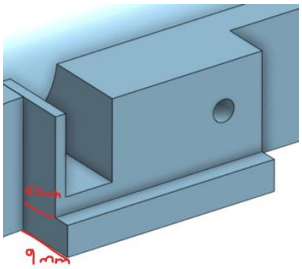


Figure 5 Sensor Mount Extruded

Figure 4 and Figure 5 depict the dimension used to mount the sensors to the track using a M3 machine screw.

Wiring Overview:

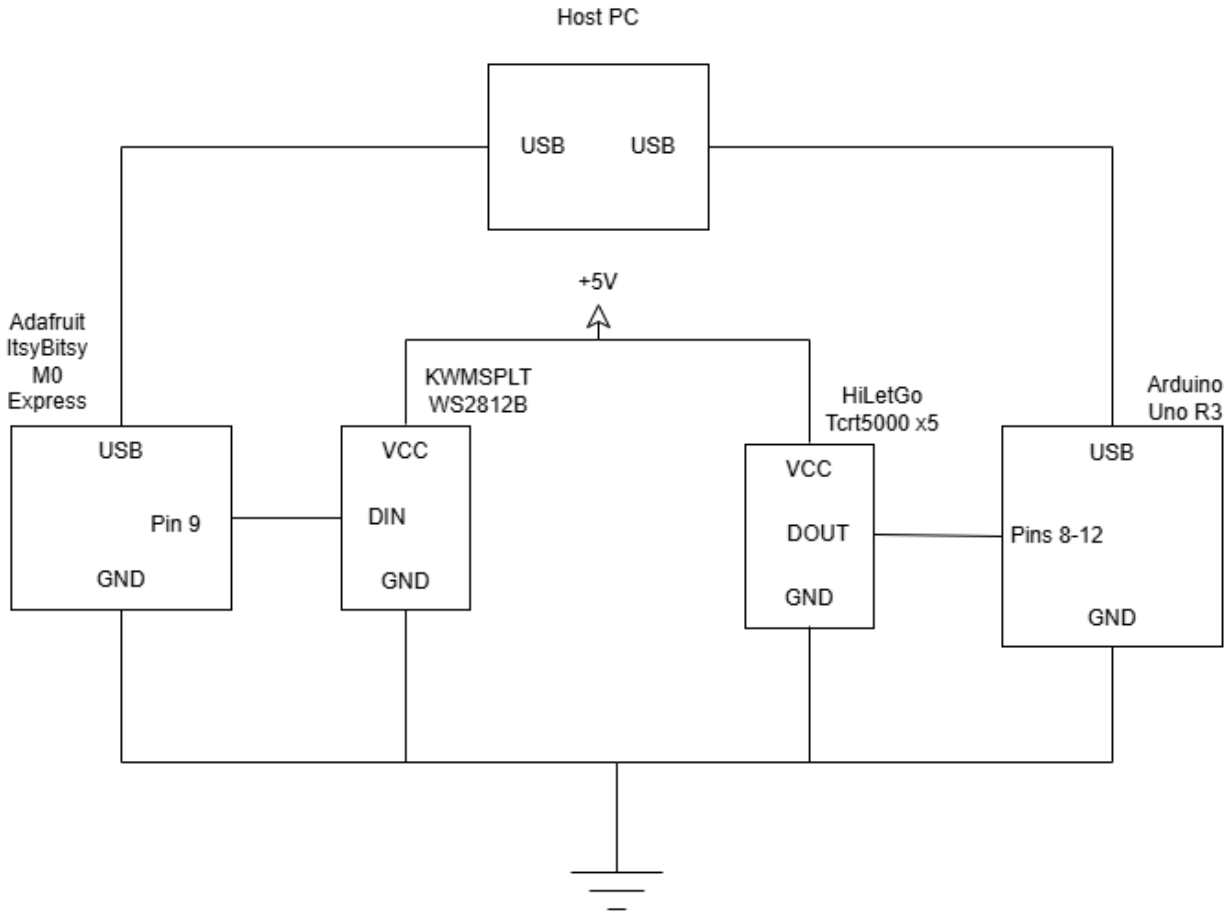


Figure 6 System Interconnect Diagram

Figure 6 depicts the overall wiring diagram for the entire electrical system that runs the sensors and the lights.

Sensor	DAQ Board pin (MCU pin)
0	8 (B0)
1	9 (B1)
2	10 (B2)
3	11 (B3)
4	12 (B4)

Table 5: DAQ GPIO Connections

Software Design Overview:

The software for this system is distributed across three devices: the DAQ microcontroller, the Host PC, and the LED Controller microcontroller. The DAQ firmware captures sensor events and transmits them to the Host PC over USB serial. The Host PC software receives this data, processes it through the machine learning model, generates a timed LED activation schedule, and transmits the resulting LED states to the LED Controller. The LED Controller firmware receives LED color data and drives the WS2812B strip. The following subsections describe the design of each software component.

DAQ Firmware

The DAQ firmware runs on an ATmega328P (Arduino Uno) and is responsible for detecting beam-break events and reporting them to the Host PC with microsecond-resolution timestamps. The firmware is written as bare-metal C++ with direct register access rather than using the Arduino IDE framework, as precise control over timer and interrupt configuration was necessary to achieve the most accurate possible timestamps. The firmware is organized into three classes: TimerManager, IRSensor, and SerialHandler.

TimerManager configures and manages the ATmega328P's 16-bit Timer1 peripheral with a prescaler of 8, yielding a 2MHz tick rate (0.5µs resolution). It tracks timer overflows via an interrupt service routine to extend the effective range beyond the 16-bit counter limit, producing a 32-bit timestamp by combining the overflow count with the current counter value. When a pin-change interrupt fires, TimerManager immediately snapshots the counter value to minimize jitter between the actual event and the recorded timestamp.

IRSensor represents a single beam-break sensor. Each instance is bound to a specific GPIO pin on PORTB and maintains its own beam status (broken or intact), break start time, and break duration. When the pin-change interrupt fires, the ISR reads the entire port state once and passes it to each IRSensor instance, which determines whether its specific pin changed. If a transition occurred, the instance applies a software debounce filter (4000 timer ticks, approximately 2ms) to reject electrical noise. On a valid beam-break, the instance records the timestamp. On beam restoration, it computes the break duration. Each measurement is flagged as ready to send, and the main loop checks these flags and packages the data into the transmit buffer.

SerialHandler manages UART communication at 115200 baud using a 256-byte circular transmit buffer. It implements the custom framed protocol described in the Design Alternatives section, packaging each sensor reading into a structured message with STX/ETX delimiters, message type, sensor ID, descriptor (start time or duration), payload length, and a 4-byte big-endian payload. The main loop drains the buffer one byte at a time between sensor checks, ensuring that serial transmission does not block sensor event detection.

LED Controller Firmware

The LED Controller firmware runs on an Adafruit ItsyBitsy M0 Express using the Arduino IDE and the FastLED library. It listens for incoming framed messages over USB serial using the same protocol as the DAQ. On receiving a command message, it dispatches to the appropriate handler based on the descriptor byte. The primary operating mode receives a full 432-byte RGB buffer (144 LEDs × 3 bytes per LED) from the Host PC and writes it directly to the LED array using FastLED. The firmware also supports individual LED commands (single LED on, toggle, all on, all off) for testing and debugging. After processing a command, the LED Controller sends an ASCII acknowledgment message back to the Host PC for display in the GUI.

Host PC Software

The Host PC software is written in Python and serves as the central processing hub of the system. It receives sensor data from the DAQ, runs the machine learning model to predict ball position, generates LED activation schedules, and transmits LED state updates to the LED Controller. The software is multithreaded, with separate threads handling serial I/O, message processing, prediction scheduling, LED animation, and the GUI (graphical user interface). Figure 8 shows the class architecture using a UML class diagram.

SerialHandler extends the pyserial Serial class and manages communication with a single serial device. Each SerialHandler instance spawns two threads: a reader thread that continuously drains incoming bytes from the UART into a raw byte queue, and a parser thread that consumes the raw bytes, identifies framed messages by their STX/ETX delimiters, and parses them into Python dictionaries containing the message type, sensor ID, descriptor, payload, and a host-side timestamp. Parsed messages are placed onto a shared global message queue. The system instantiates two SerialHandler objects, one for the DAQ and one for the LED Controller, both feeding the same shared queue. SerialHandler also provides a send_frame method used by other modules to transmit commands to the LED Controller.

MessageProcessor is the central routing and data management module. Its inbound worker thread pulls parsed messages from the shared queue and routes them to handler functions based on the message source and type using a dictionary-based dispatch table. For DAQ data messages, it delegates validation and pairing to a Run object. The Run class represents a single ball traversal of the track, managing the lifecycle of one run from the first sensor trigger to the last. It performs software debouncing, enforces sensor ordering to reject stale data from sensors the ball has already passed, and pairs incoming start-time and duration messages for the same sensor into a complete data record. When a complete sensor record is assembled, MessageProcessor submits it to PredictionPlanner and writes it to the log file. A watchdog thread monitors for stale runs (no activity for 10 seconds) and resets the system if a run appears to have been abandoned. After the final sensor in the run fires, MessageProcessor ends the run, cancels any pending LED timers, and begins a cooldown period to prevent immediate re-triggering from ball bounce.

PredictionPlanner receives completed sensor data records and generates LED activation schedules. Its worker thread calls the trained scikit-learn linear regression model (loaded from a pickled file at startup) to predict the timestamp at which the ball will reach the next sensor. Using

the known physical distances between sensors and LEDs (defined in the CONST module), it computes the estimated time for the ball to reach each LED between the current sensor and the next sensor, assuming constant velocity within each sensor-to-sensor segment. These times are packaged into a schedule dictionary mapping LED indices to activation delay times. The schedule is passed to the MessageProcessor outbound worker, which creates a Python threading.Timer for each LED activation. When each timer fires, it calls the Animator to update the corresponding LED in the buffer.

Animator maintains an RGB color buffer representing the state of all LEDs on the strip. A dedicated worker thread periodically transmits this full buffer to the LED Controller at a configurable refresh rate (120Hz by default). This periodic refresh approach, as described in the Design Alternatives section, decouples the animation logic from the communication logic. The Animator supports multiple animation styles through an AnimationStyle abstract base class, allowing different visual effects (such as a trailing fill or rainbow coloring) to be selected at runtime through the GUI without modifying the underlying data processing or communication code.

The Coordinator class serves as the integration layer between the processing backend and the graphical user interface. It uses PySide6 Qt signals and slots to safely bridge the multithreaded backend with the single-threaded GUI event loop. The GUI (MainWindow) provides a soft front panel with three text viewing panels: a CSV log viewer, a detailed log viewer, and a command/error reporting panel, along with an LED strip visualizer that mirrors the current state of the physical strip, and control buttons for LED commands, animation style selection, and trial management. Figure 7 shows the GUI in “SFP Debug” mode as it appears when the Host PC program is run.

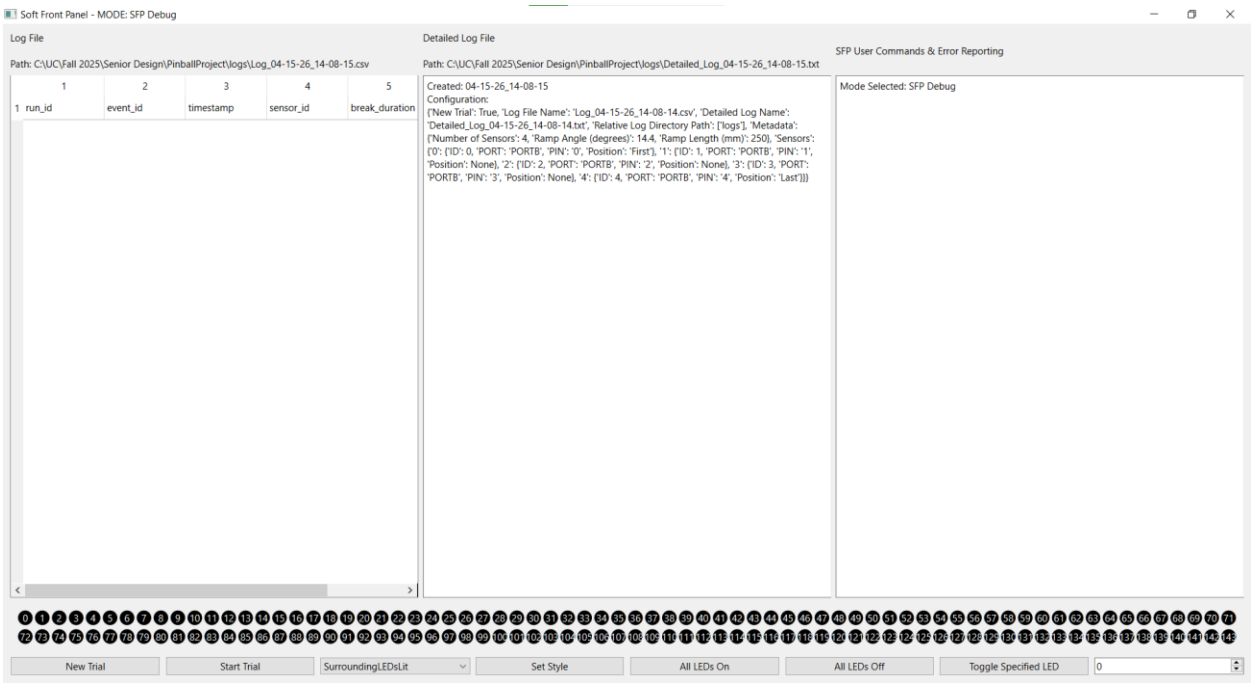


Figure 7: Soft Front Panel GUI

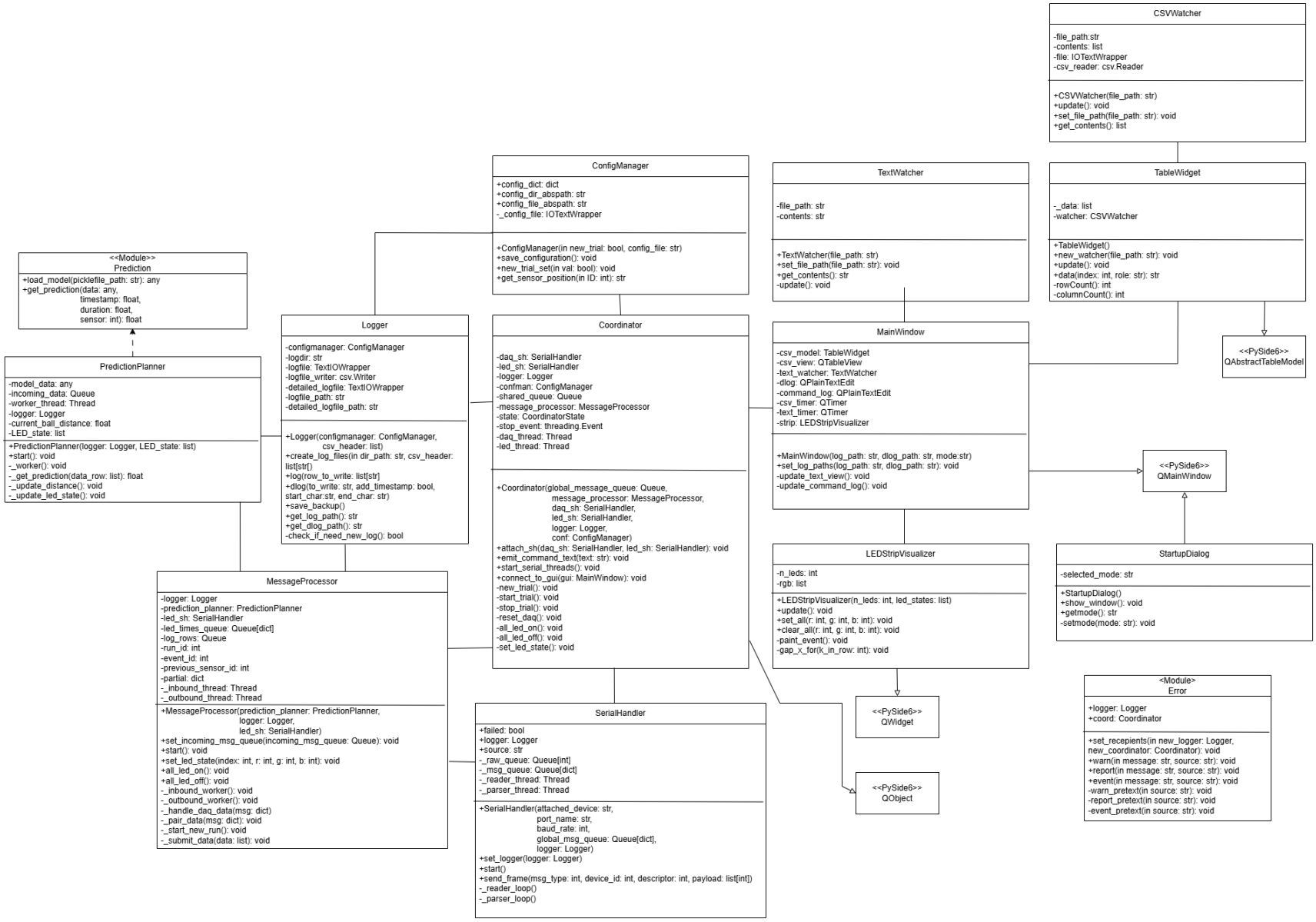


Figure 8 Host PC UML Class Diagram

Figure 9 shows the logical data flow through the system, from initial sensor detection through machine learning prediction to LED activation. Each process represents a transformation of data, and each arrow indicates the form of the data at that stage of the pipeline.

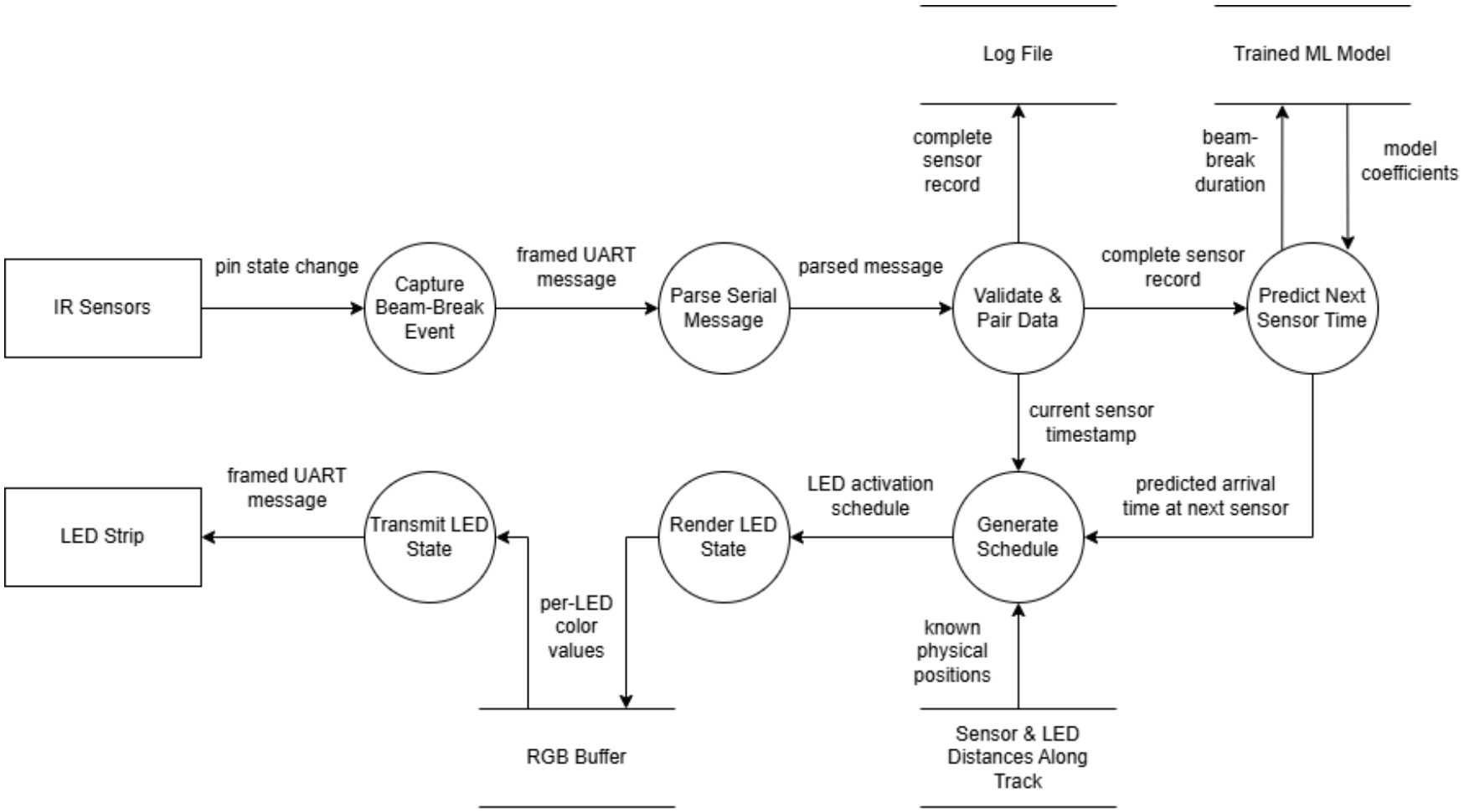


Figure 9: System Logical Data Flow Diagram

Machine Learning Model

The machine learning model utilized a 2 parameter equation fit to a linear regression model. The break duration of the sensors was fed into the model, and the predicted duration of time before the next sensor is broken was predicted by the model. This prediction would then be used for further processing in order to generate a schedule for LED illumination.

$$\Delta T_n^* = c_0 + c_1 \Delta t_n$$

Where

ΔT_n^* : Predicted time between sensor n and n+1

Δt_n : Break duration of sensor n

c_0 and c_1 : Constants generated from Machine Learning Model

Figure 10 Machine Learning Model Equation Fitting

After the model had been fitted to the linear regression model, the model was required to be validated with the data withheld from training. Upon using the test data in the model, 3 values could be generated to asses model performance. These were the R-squared value, Mean absolute error, and root mean square error. The assessed performance of the model generated between each sensor in the 5 sensor version of the track is as follows:

Sensor 0 to Sensor 1	R^2 Score: 0.9632 RMSE: 0.011183 seconds MAE: 0.009737 seconds
Sensor 1 to Sensor 2	R^2 Score: 0.9697 RMSE: 0.006728 seconds MAE: 0.004562 seconds
Sensor 2 to Sensor 3	R^2 Score: 0.9987 RMSE: 0.003775 seconds MAE: 0.002988 seconds
Sensor 3 to Sensor 4	R^2 Score: 0.9998 RMSE: 0.001689 seconds MAE: 0.001464 seconds

Prototype and/or Product Development and Testing [ABET 2, 3, 6]

Pre-Manufactured Components

The system uses two pre-manufactured microcontroller development boards as its primary hardware components. The Arduino Uno R3, based on the ATmega328P [2], was selected for the DAQ module. This board was chosen because its GPIO pins support pin change interrupt vectors capable of servicing all beam-break sensors simultaneously. Its 16-bit Timer1 peripheral, configured with a prescaler of 8, provides a 2MHz tick rate (0.5µs resolution) for measuring beam-break timestamps and durations with the precision required by the machine learning model. The hardware UART peripheral supports communication used to transmit data to the Host PC.

The Adafruit ItsyBitsy M0 Express, based on the SAMD21G18A microcontroller [4], was selected for the LED Controller module. Its 48MHz clock speed provides sufficient processing headroom to receive and parse serial commands while simultaneously driving the LED strip. The ItsyBitsy M0 Express board was selected specifically for its compact form factor, which is suitable for mounting alongside the LED strip inside a pinball machine ramp enclosure. The LED Controller communicates with the Host PC over its native USB serial interface using the same framed protocol as the DAQ module.

The HiLetGo TCRT5000 [8] infrared sensors were used for ball detection. Five sensors were mounted along the prototype track as described in the Track Manufacturing section. These sensors were selected for their digital output, low cost, and compact form factor, though their sensitivity to jostling of the track and inconsistent triggering at high ball speeds presented challenges during development that required the addition of software debouncing in the DAQ firmware and more data validation checks in the Host PC software.

The WS2812B addressable LED strip [1] was selected for the visual output. This strip was chosen because each LED is individually addressable over a single data line, eliminating the need for additional shift registers or GPIO pins on the LED Controller.

Software Libraries

The machine learning model was developed using the Sklearn library [1][5] in python. This library was utilized due to its ability to train a linear regression model to selected features in the data in a quick and repeatable way. The ability to measure the performance of the model is a key factor in the development. Therefore, only 80% of the data collected was used for training the model. The other 20% was used as test inputs in order to validate the models performance. Additionally, Pandas and Numpy packages were utilized for data handling purposes.

The WS2812B LED strip is driven using the FastLED Library [6] which handles the timing-critical pulse widths required by the WS2812B protocol. FastLED was selected over a bare-metal

DMA implementation (which was developed during initial prototyping) for its reliability and ease of maintenance. The LED Controller firmware was developed in the Arduino IDE to take advantage of FastLED's integration with that environment
Serial communication between the Host PC and both microcontrollers was handled using the pyserial library [7]. Pyserial was selected because it provides cross-platform access to serial ports from Python and supports the byte-level read and write operations required by the custom framed UART protocol.

The GUI was built using PySide6 [9], the official Python binding for the Qt framework. PySide6 was selected because its signals and slots mechanism provides a thread-safe way to update GUI elements from multiple backend threads handling serial communication, message processing, and LED animation.

Track Manufacturing

All tracks were produced with a Hobbyist FDM 3D printer. The tracks were designed to be assembled using dovetail connectors and the sensors were mounted with M3 machine screws.

Prototype Track 1:

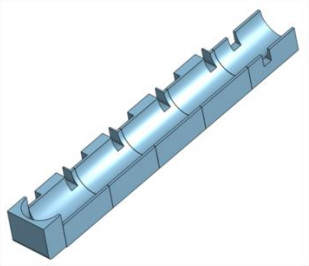


Figure 11 Straight Track

A straight track was used to test the sensors and determine the most effective way to capture and transmit readings. This design was chosen because the ball's speed at each sensor could be calculated mathematically, allowing the measured values to be validated against expected results.

In constructing this track and testing the sensor readings, it was concluded that the sensors could accurately process the ball's speed and location quickly enough to run the lights along the track.

Prototype Track 2:

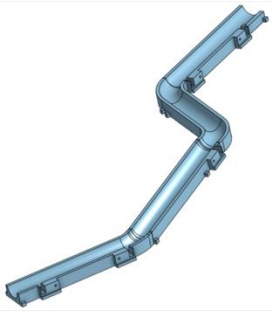


Figure 12 Complex Track

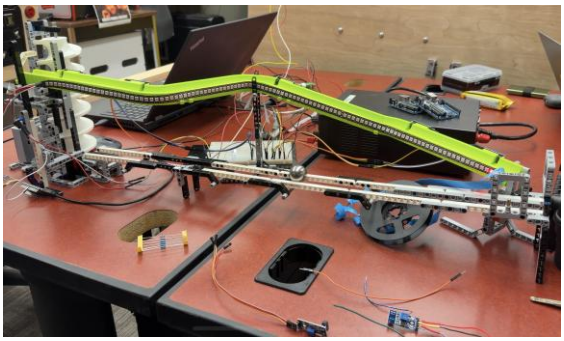


Figure 13 Pinball Relay System

Next, a complex track was developed to replicate a real-world pinball machine. Using Lego Technic, a pinball relay system (see Figure 133) was created to automate data collection for training a machine learning model. This system used a 3D-printed corkscrew to lift the ball back to the top of the track after each run. The track height was adjustable, allowing for a larger range of initial ball speeds, expanding the data pool and improving the model. This track was also where the LED controller was introduced, using an open-source library to drive the LED strip.

Pinball Cabinet Integration

To enhance the display of the lighting system, it was integrated into a full-sized pinball cabinet supplied by project sponsor. The cabinet consisted of a working flipper system, which could be used to shoot the ball onto the ramp containing the lighting display. The functionality of the lighting system in the physical cabinet was largely unchanged from that of the relay track setup shown in Figure 13.

The flipper system required additional fabrication and wiring. A standard MOSFET driver circuit was utilized to accept PWM inputs at a higher voltage for the flippers exhibited in Figure 14. A custom circuit board was soldered to achieve this desired circuit shown in Figure 15. PWM was handled by an Arduino uno using standard flipper control code.

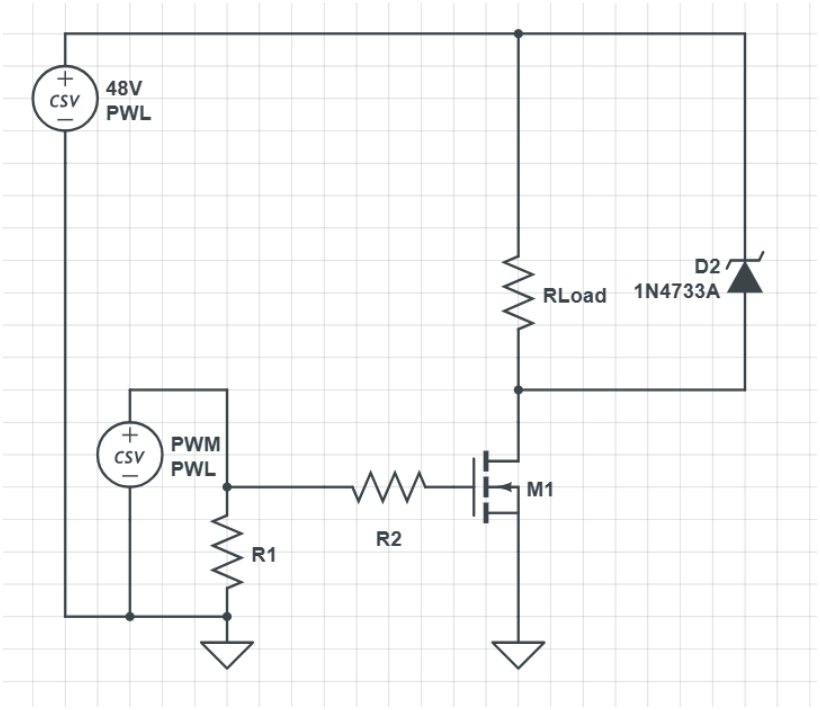


Figure 14 MOSFET Driver Circuit Schematic

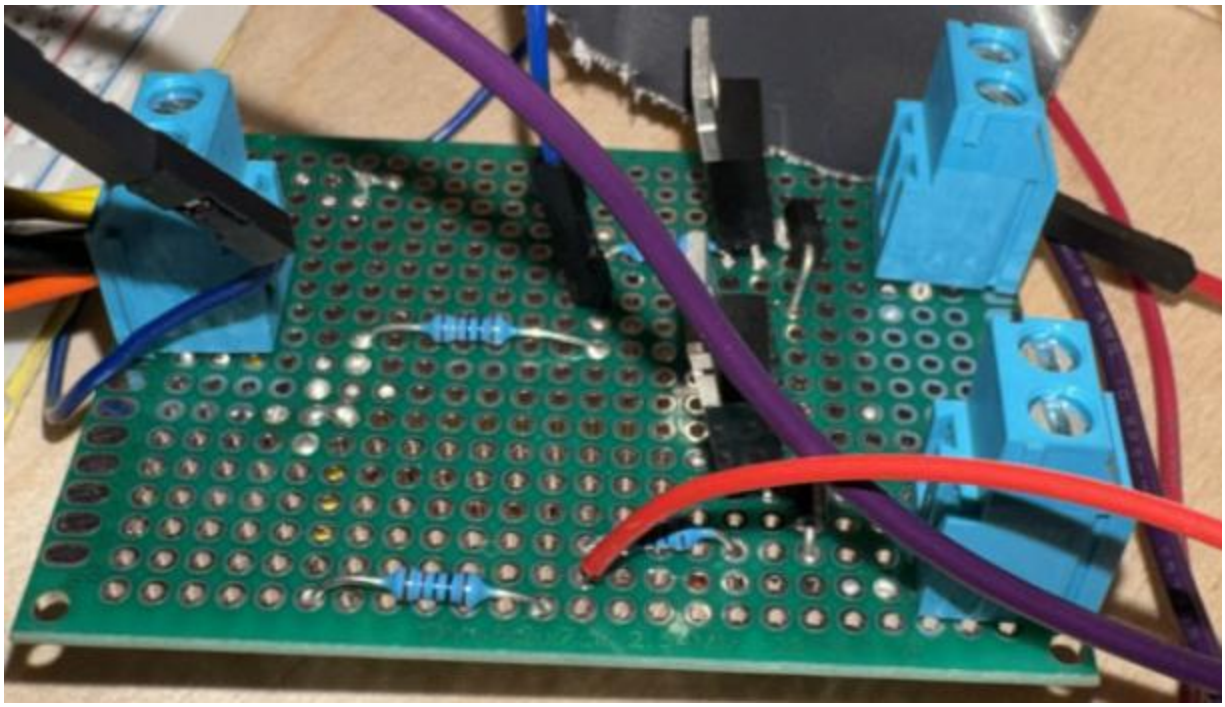


Figure 15 Soldered MOSFET Driver Circuit

Test Criteria [ABET 2]

The test will consist of monitoring a ball traversing the ramp using a slow-motion camera. The difference between the LED strip position and the actual position of the ball will be measured. This error will be recorded in the number of LEDs and then converted into an actual distance.

Requirement	**Test Procedure**	**Type**	**Data Gathered**
Accurate Sensor Signal Detection	Signal Capture Test	Verification	Oscilloscope vs. DAQ signal detection comparison (±500µs tolerance)
LED Functionality	LED Functionality Test	Verification	Confirmation of basic LED Controller functionality.
Accuracy of Activated LED Compared to Ball Location	LED Accuracy Test	Validation	Accuracy of activated LED to pinball location (±2cm tolerance)
System Latency	System Latency Test	Validation	End-to-end system latency. (<20ms)

Table 6: Requirements Traceability Matrix

Test Plan [ABET 6]

This section lays out multiple necessary test plans that were conducted for this project and the details for each of the tests.

Signal Capture Test Procedure

The purpose of the Signal Capture Test is to verify the DAQ system is functioning properly, specifically that the outputs of the IR sensors match what the DAQ reports.

Test Equipment:

- Oscilloscope
- DAQ MCU (Arduino Uno)
- IR/Beam-break Sensors x5
- Pinball
- Test Track
- Host PC

Prerequisites:

Before beginning the Signal Capture Test, the following prerequisites must be met:

- The DAQ MCU must be programmed.
- The Host PC software must be downloaded.
- Sensors 0-4 must be mounted to the test track.
- Sensors 0-4 must be connected to +5V and ground, common with the ground of the DAQ MCU.
- Sensors 0-4 must have their digital outputs connected to the DAQ MCU, following the setup as detailed in **Error! Reference source not found.**.

Test Steps:

Step	Action	Pass Criteria
1	Connect the DAQ MCU to the host PC via the DAQ MCU USB type B connector and any USB type A connector on the Host PC	N/A
2	Power on the DC 5V power supply. Read the value on the power supply display.	5V ± 0.1V
3	Run /PinballProject/python/main.py, select “DAQ Only” when the pop-up menu appears	“Ready!” ASCII message displayed in SFP User Commands & Error Reporting text box.
4	Connect an oscilloscope probe to the digital output of Sensor 0. Read the value displayed on the oscilloscope	5V ± 0.5V
5	Send the pinball down the test track from the beginning. Use the oscilloscope to measure the time between the falling edge and rising edge of the sensor output (Use 50% of the maximum sensor digital output voltage to find	Oscilloscope value ± 500µs of DAQ reported value.

	the edges) Compare this to the “break_duration” value reported by Sensor 0 in the Log File text box	
6	Repeat steps 4 & 5 for each sensor on the test track.	N/A

LED Functionality Test Procedure

The purpose of the LED Test is to verify the LED Controller module is functional, including the ability of the LED Controller MCU to command the WS2812B LED strip, and the functionality of each LED on the strip.

Test Equipment:

- LED Controller MCU (Adafruit ItsyBitsy M0 Express)
- Test Track
- WS2812B LED Strip
- Host PC

Prerequisites:

Before beginning the LED Functionality Test, the following prerequisites must be met:

- The LED Controller MCU must be programmed.
- The Host PC software must be downloaded.
- The WS2812B LED strip must be mounted on the test track.
- The WS2812B LED strip must be connected to +5V and ground, common with the ground of the LED Controller MCU
- The data in pin of the WS2812B LED strip must be connected to pin 9 of the LED Controller MCU

Test Steps:

Step	Action	Pass Criteria
1	Connect the LED Controller MCU to the host PC via the LED Controller MCU micro USB connector and any USB type A connector on the Host PC	N/A
2	Power on the DC 5V power supply. Read the value on the power supply display.	5V ± 0.1V
3	Run /PinballProject/python/main.py, select “LED Controller Only” when the pop-up menu appears	“Ready!” ASCII message displayed in SFP User Commands & Error Reporting text box.
4	Click the “All LEDs On” button on the SFP. Ensure all LEDs are on through visual confirmation.	All LEDs have turned on.
5	Click the “All LEDs Off” button on the SFP. Ensure all LEDs are off through visual confirmation.	All LEDs have turned off.

System Latency Test Procedure

The purpose of the System Latency Test is to validate the performance of the system, ensuring the end to end system latency requirement of less than 20ms is met.

Test Equipment:

- Test Track
- Pinball
- LED Controller MCU (Adafruit ItsyBitsy M0 Express)
- DAQ MCU (Arduino Uno)
- IR/Beam-break Sensors x5
- Host PC

Prerequisites:

- The prerequisites of both the Signal Capture Test and the LED Functionality Test (excluding the WS2812B LED strip prerequisites) must be met, ensuring the full system is assembled.

Test Steps:

Step	Action	Pass Criteria
1	Connect the LED Controller MCU to the host PC via the LED Controller MCU micro USB connector and any USB type A connector on the Host PC	N/A
2	Connect the DAQ MCU to the host PC via the DAQ MCU USB type B connector and any USB type A connector on the Host PC	N/A
3	Power on the DC 5V power supply. Read the value on the power supply display.	5V ± 0.1V
4	Run /PinballProject/python/main.py, select “Latency Test” when the pop-up menu appears	“Ready!” ASCII message displayed in SFP User Commands & Error Reporting text box.
5	Send a pinball down the ramp. Ensure sensor readings are reported to the SFP.	Sensor readings are reported in the Log File text box.
6	Record the “Average Latency” value displayed in SFP User Commands & Error Reporting text box.	N/A
7	Repeat steps 5 and 6 ten times, and calculate the average of the average latency values	<20ms

LED Accuracy Test Procedure

The purpose of the LED Accuracy Test is to validate the performance of the system, ensuring the requirement that the LEDs remain within 2cm of the pinball’s location at all times during a run is met. LED lengths are used as the unit of measurement during the test, as the left edge of each LED is ~6.5mm apart. Therefore an accuracy of <3 LED lengths (19.5mm) meets the requirement of 2cm accuracy.

Test Equipment:

- Test Track
- Pinball
- LED Controller MCU (Adafruit ItsyBitsy M0 Express)
- DAQ MCU (Arduino Uno)
- IR/Beam-break Sensors x5
- WS2812B LED Strip
- Host PC
- 120FPS Camera (iPhone or similar)

Prerequisites:

- The prerequisites of both the Signal Capture Test and the LED Functionality Test must be met, ensuring the full system is assembled.

Test Steps:

Step	Action	Pass Criteria
1	Connect the LED Controller MCU to the host PC via the LED Controller MCU micro USB connector and any USB type A connector on the Host PC	N/A
2	Connect the DAQ MCU to the host PC via the DAQ MCU USB type B connector and any USB type A connector on the Host PC	N/A
3	Power on the DC 5V power supply. Read the value on the power supply display.	5V ± 0.1V
4	Run /PinballProject/python/main.py, select “Full System” when the pop-up menu appears	“Ready!” ASCII message displayed in SFP User Commands & Error Reporting text box.
5	Select “SingleLED” animation style and click “Set Style"	“Animation style set to: SingleLED” appears in SFP User Commands & Error Reporting text box.
6	Hold the camera approximately 3ft above the midpoint of the ramp, ensuring the entirety of the ramp is in frame.	N/A
7	Start recording and send the pinball down the ramp.	LEDs are activated during the run.
8	Review the recording frame-by-frame, comparing the location of the ball to the location of the activated LED. Record the maximum number of LED lengths between the activated LED and the closest edge of the ball.	The activated LED is always < 3 LED lengths from the closest edge of the pinball during the run.

Test Results [ABET 3]

Test	Result	Pass/Fail
Signal Capture Test	Sensor 0 difference: 62.5µs Sensor 1 difference: 85.5µs Sensor 2 difference: 181µs Sensor 3 difference: 63µs Sensor 4 difference: 22µs	Pass
LED Functionality Test	All LEDs turned on & off when commanded.	Pass
System Latency Test	Average Latency: 128.39ms	Fail
LED Accuracy Test	Ball remained within 3 LED lengths of activated LED during duration of run.	Pass

The system failed the System Latency Test, however the latency value gathered during this test was used as an offset for the LED activation times, allowing the system to function without visual latency and pass the LED Accuracy Test.

Social Impact [ABET 2, 4]

Predictive modeling can improve systems by reducing the amount of hardware required while maintaining acceptable performance levels. By developing a light prediction model that accurately predicts the ball's position as it moves down the ramp, this approach demonstrates how that can be achieved.

The concept of reducing hardware costs through predictive modeling can be applied to other system designs as well, making technology more accessible to diverse groups. Applications in healthcare, logistics, and automation could enable companies to offer a wider range of products and services at lower costs, allowing more people to benefit from the technology.

Future Work and Subsequent Development [ABET 7]

The primary goal of this project was to use a machine learning model to reduce the number of sensors along the track. While this goal was not fully achieved, the foundation has been established to refine the system and decrease the number of sensors required.

Further testing and troubleshooting would be necessary to accomplish this. Ideally, only one sensor would be needed at the start of the track, though additional testing is required to confirm this. At most, an additional sensor may be needed at the end of the track.

Although not part of the original scope, future work could focus on refining the electronic hardware to be more user-friendly, potentially enabling a plug-and-play design for easier integration into pinball machines. Additionally, many code improvements could be made, including code refactoring for robustness/system latency, full implementation of the SFP trial

control buttons, and rewriting of the LED Controller code to be bare-metal, using the DMA peripheral for the WS2812B control, resulting in reduced system latency.

Professor Fuchs already has designs for pinball tracks that have built-in LED strips. Another project continuation could be designing modification attachments that could be added onto standard pinball tracks to attach LEDs and sensors to them.

If this project were to revisited, an investment into higher quality sensors would help mitigate the reliability challenges encountered with the low-cost sensors used.

Summary and Conclusions [ABET 3]

The result of this project was a fully functioning light system that can accurately follow alongside a pinball as it goes down the track using trained data from a machine learning model and beam break sensors along the length of the track.

This project includes several features, such as a variety of light animations and a GUI system that allows users to toggle animations, test LED strip lights, and read sensor input. It also includes CAD models of the tracks used.

This project still uses the full number of sensors along the track. The goal of reducing the number of sensors to one or two was not achieved, but the foundation is in place to refine the system and reduce the sensor count.

The current system failed the System Latency Test and therefore did not meet the system latency requirement. However, the information gathered during this test allowed for a workaround to be made in order to still meet the LED accuracy requirement and allow for smooth visuals during gameplay.

Team Design Reflection and Debriefing [ABET 5]

1. List the most powerful learning moments or experiences you encountered. Why do you feel they were profound and how do you think they might shape your future approaches to problems?

   The most powerful learning moment of this project involved being given a vague set of requirements and having to make a successful product out of it. Our team was required to conceptualize a project ourselves and bring it to life. Almost all the other work that was required for completing our degrees in mechanical and electrical engineering consisted of structured problems with outlined requirements. This was great for developing our problem-solving intuition but had little practicality outside of the classroom. Senior design required us to identify a problem on our own and build a solution from the ground up. This is very similar to how we will be required to solve problems in the workplace. Often, managers will not be able to lay out every constraint needed to arrive at a solution, but instead we as employees will need to identify them ourselves and build our own narrative. Having completed this project, we now believe that to be successful in an engineering workplace, we should start by identifying the most important problems and developing our own roadmap to solve them. This course has provided us with an

opportunity to practice doing that for ourselves and we will now be more confident to build on this experience in the future.

2. What have you learned about supporting and encouraging the members of your team? How might these lessons shape your future approaches to engineering problem solving and building teams? [ABET 5]

   Through participating in senior design as a team, we have learned that all team members will have different motivations and personalities. Each member plays a different role in team dynamics, and they should all be recognized and appreciated. It is also important to the function of the team that there is diversity in the makeup of the team. By having both mechanical and electrical engineers, we were able to utilize our unique backgrounds to make contributions to the project. The team would not have been able to function effectively with only a singular perspective. We will utilize these lessons when approaching future engineering problems by seeking out diverse perspectives when building teams. We will also welcome varying personalities, even if different than our own, to enhance the effectiveness of the team. Welcoming those voicing dissenting opinions, while initially frustrating, can help a team arrive at a better and more complete solution which we have experienced at times during this project.

3. What did you discover to be your team's greatest strengths? What did you discover to be your team's greatest weaknesses? What could you have done to ameliorate the weaknesses and further enhance the strengths?

   We discovered that our team's greatest strength is that each individual member was dedicated to following through with their portion of the project. We each had our own strengths that could be contributed towards the project and were excited to work towards developing it. Our greatest weakness is that we did not always clearly communicate what would be expected from each member. This led to us sometimes going off in our own direction and causing some inefficiencies in the development. However, this weakness had some unintended benefits as well, where new ideas and breakthroughs were achieved. We could have ameliorated these weaknesses by communicating better about what our specific goals were rather than being vague.

4. All team members would have at this point engaged in multiple co-op experiences. Did you bring any lessons from practical co-op work into this project? Did you bring any lessons from this project into ongoing co-op or other real-world work? If so, what were those lessons and how did they provide you with skills and aptitudes you might not otherwise have possessed?

   Having worked in professional environments prior to beginning this project, our team members brought about a sense of professionalism that is not typical of student project groups. We understood the importance of interactions between teammates and already knew how to do it effectively from the workplace. Additionally, we knew the types of problems that engineers typically focus on out in the field. We applied the thought

processes that we learned from our workplaces to this project which allowed us to complete it in a more professional manner.

5. What advice would you give to future students to help them do well in senior team design? Note: this is advice about how to do well in the class and get the most from it as opposed to advice on how to achieve the engineering product outcomes similar to yours.

   The advice we would give to other teams to be successful in senior design is to be proactive when taking steps to complete the project. It is best to do most of the heavy lifting for the project up front rather than waiting until the last minute. This will give you time to focus on polishing up the project at the end of the semester rather than trying to make it work. By doing this, you are able to focus more on how you want to present the project and decide which narrative you want to tell regarding it. Creating a story around the project you are making is almost as important as the technical specifications of the project itself. This is how you will get people to notice and appreciate your project when presenting it at expo, and it is also how you will be able to sell yourself and your work to a future employer.

References [ABET 7]

[1] Worldsemi, "WS2812B Single-line 256 Gray-level 3-channel Constant Current LED Driver IC." Worldsemi, Dongguan, Guangdong, China, 2025

[2] Atmel, "8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash." Atmel Corporation, San Jose, CA, 2015

[3] "Linear regression example," Scikit Learn, https://scikit-learn.org/1.5/auto_examples/linear_model/plot_ols.html (accessed Feb. 9, 2026)

[4] Microchip, "Low-Power, 32-bit Cortex-M0+ MCU with Advanced Analog and PWM SAM D21/DA1 Family." Microchip Technology, Chandler, AZ, 2025

[5] "API Reference," *scikit-learn*. https://scikit-learn.org/stable/api/index.html

[6] "FastLED: FastLED," *fastled.io*. https://fastled.io/docs/

[7] "pySerial API — pySerial 3.4 documentation," *pyserial.readthedocs.io*. https://pyserial.readthedocs.io/en/latest/pyserial_api.html

[8] "Reflective Optical Sensor with Transistor Output TCRT5000, TCRT5000L Vishay Semiconductors" Available: https://www.vishay.com/docs/83760/tcrt5000.pdf

[9] "Qt for Python — Qt for Python," *doc.qt.io*. https://doc.qt.io/qtforpython-6/

Appendix A: Source Code

DAQ (C++):

main.cpp

```
/*
 * main.cpp
 *
 * Created: 10/2/2025 1:03:58 PM
 * Author : Noah Sergent
 *
 *
 */

#define F_CPU 16000000UL

#include <avr/interrupt.h>
#include <avr/io.h>
#include <stdint.h>

#include "include/IRSensor.h"
#include "include/SerialHandler.h"
#include "include/TimerManager.h"

void packageSensorInfo( IRSensor *sensor, uint8_t
payload_holder[IRSensor::RECORD_SIZE] );
void bytesFrom32BitInt( uint32_t val, uint8_t bytes[4] );
void sendReady();

const uint8_t pcmsk0 = ( 1 << PCINT0 ) | ( 1 << PCINT1 ) | ( 1 << PCINT2 ) | ( 1
<< PCINT3 ) | ( 1 << PCINT4 );

const uint8_t ubbrl = 8;
const uint8_t ubbrh = 0;

uint8_t ready_msg[] = "Ready!";
uint8_t run_finished_msg[] = "Run finished";
uint8_t message_failed_msg[] = "Message Failed!";
uint8_t tx_overflow_msg[] = "tx Buffer Overflow!";

volatile bool run_finished = false;
volatile uint8_t temp_port_val;
```

```
IRSensor sensor0( 0, PINB, PINB0, SensorPosition::First ); // TODO implement
command to define sensors at runtime
IRSensor sensor1( 1, PINB, PINB1 );
IRSensor sensor2( 2, PINB, PINB2 );
IRSensor sensor3( 3, PINB, PINB3 );
IRSensor sensor4( 4, PINB, PINB4 );

const uint8_t NUM_SENSORS = 5;
IRSensor *sensor_array[] = {
    &sensor0,
    &sensor1,
    &sensor2,
    &sensor3,
    &sensor4
};

SerialHandler::MsgStruct message;
uint8_t payload_holder[IRSensor::RECORD_SIZE];

int main( void ) {
    SerialHandler::init( ubbrl, ubbrh );
    TimerManager::configure( PrescalerValue::N_8 );

    DDRD |= ( 1 << PIND7 );

    sendReady();
    for ( uint16_t count; count < 10000; count++ )
        __asm__( "nop" );
    IRSensor::configureInterrupts( pcmsk0 );

    while ( 1 ) {
        SerialHandler::sendByteFromBuffer();          // Clear out TX buffer a
byte at a time.
        for ( uint8_t i = 0; i < NUM_SENSORS; i++ ) { // Check each sensor and
queue pending data
            packageSensorInfo( sensor_array[i], payload_holder );
        }
        // if(IRSensor::send_finished_msg_flag) {                   // If the
last sensor was triggered, send message so python creates new csv
        //  IRSensor::send_finished_msg_flag = false;
        //  sendReady(run_finished_msg);
        // }
    }
}
```

```
void sendReady() {
    const uint8_t ready_len = sizeof( ready_msg ) - 1;

    message = SerialHandler::msgPackager(
        SerialHandler::MESSAGE,
        0xFF,
        0xFF,
        sizeof( ready_msg ),
        ready_msg );
    SerialHandler::enqueueStructuredMsg( &message, ready_len + 6 );
    SerialHandler::flushBuffer();
}

void packageSensorInfo( IRSensor *sensor, uint8_t
payload_holder[IRSensor::RECORD_SIZE] ) {

    if ( !( sensor->start_time_ready_to_send_ || sensor->duration_ready_to_send_
) ) {
        return;
    }

    uint8_t descriptor;
    bool send = false;

    if ( sensor->start_time_ready_to_send_ ) {

        descriptor = sensor->START_TIME_DESCRIPTOR;
        bytesFrom32BitInt( sensor->getStartTime(), payload_holder );
        sensor->start_time_ready_to_send_ = false;
        send = true;
    } else if ( sensor->duration_ready_to_send_ ) {

        descriptor = sensor->DURATION_DESCRIPTOR;
        bytesFrom32BitInt( sensor->getDuration(), payload_holder );
        sensor->duration_ready_to_send_ = false;
        send = true;
    }

    if ( send ) {
        message = SerialHandler::msgPackager(
            SerialHandler::DATA,
            sensor->sensor_id_,
            descriptor,
            sensor->RECORD_SIZE,
            payload_holder );
```

```
        SerialHandler::enqueueStructuredMsg( &message, IRSensor::RECORD_SIZE + 6
);
    }
}

void bytesFrom32BitInt( uint32_t in, uint8_t *out ) {
    out[0] = ( uint8_t )( in >> 24 );
    out[1] = ( uint8_t )( in >> 16 );
    out[2] = ( uint8_t )( in >> 8 );
    out[3] = in;
}

ISR( PCINT0_vect ) {
    temp_port_val = PINB;
    TimerManager::saveTimestamp();
    for ( uint8_t i = 0; i < NUM_SENSORS; i++ ) {
        if ( sensor_array[i]->wasTriggered( temp_port_val ) ) {
            sensor_array[i]->recordBreakEvent();
            if ( i == 4 ) {
                PORTD ^= ( 1 << PORTD7 );
            }
        }
    }
}
```

TimerManager.h

```
/*
TimerManager.h

Author: Noah Sergent
Date: 10/24/25

This class manages a timer, in the current implementation, this is only timer1 on the atmega328p
*/

#ifndef TIMERMANAGER_H
#define TIMERMANAGER_H

#include<stdint.h>

enum class PrescalerValue: uint8_t {
  N_1 = 0,
  N_8 = 1,
```

```
    N_64 = 2,
    N_256 = 3,
    N_1024 = 4,
    STOP = 5
};

enum class TimerStatus: uint8_t {Running, Stopped};

class TimerManager {
    public:
        TimerManager();
        static void configure(PrescalerValue N);
        static void configure(PrescalerValue N, uint16_t ctc_value);
        static void startTimer1(bool reset);
        static void stopTimer1();
        static void incrementOverflowCount();
        static uint32_t getTotalCount();
        static uint16_t getCount();
        static uint16_t getOverflowCount();
        static TimerStatus getTimerStatus();
        static void saveTimestamp();
        static uint32_t getTotalTimestampCount();

    private:
        static const uint8_t prescaler_bits[];
        static uint8_t saved_TCCR1B_;
        static uint16_t overflow_count_;
        static TimerStatus timer_status_;
        static uint16_t timestamp_;
        static uint16_t overflow_timestamp_;
};


#endif
```

TimerManager.cpp

```
/*
TimerManager.cpp

Author: Noah Sergent
Date: 10/28/25

Implements a class to handle the beam break sensor input. Includes functions to
measure the duration of a beam break, and the time of a beam break.
*/
```

```cpp
#include <avr/io.h>
#include <avr/interrupt.h>
#include "TimerManager.h"
#include <stdint.h>

// Defining Statics
uint8_t     TimerManager::saved_TCCR1B_         = 0;
uint16_t    TimerManager::overflow_count_       = 0;
TimerStatus TimerManager::timer_status_         = TimerStatus::Stopped;
uint16_t    TimerManager::timestamp_            = 0;
uint16_t    TimerManager::overflow_timestamp_   = 0;

const uint8_t TimerManager::prescaler_bits[6] = {
    (1 << CS10),
    (1 << CS11),
    (1 << CS11) | (1 << CS10),
    (1 << CS12),
    (1 << CS12) | (1 << CS10),
    0
};

TimerManager::TimerManager() {timer_status_ = TimerStatus::Stopped;}

void TimerManager::configure(PrescalerValue N) {
    saved_TCCR1B_ = prescaler_bits[static_cast<uint8_t>(N)];
    TIMSK1 = (1 << TOIE1);
    sei();
}

void TimerManager::configure(PrescalerValue N, uint16_t ctc_value) {
    saved_TCCR1B_ = prescaler_bits[static_cast<uint8_t>(N)] | (1 << WGM12);
    TIMSK1 = (1 << OCIE1A);
    OCR1A = ctc_value;
    sei();
}

void TimerManager::startTimer1(bool reset) {
    if(reset) {TCNT1 = 0; overflow_count_ = 0;}
    TCCR1B = saved_TCCR1B_;
    timer_status_ = TimerStatus::Running;
}

void TimerManager::stopTimer1() {
    TCCR1B = 0;
```

```
    timer_status_ = TimerStatus::Stopped;
}

TimerStatus TimerManager::getTimerStatus() {return timer_status_;}

uint16_t TimerManager::getCount() {return TCNT1;}

uint16_t TimerManager::getOverflowCount() {return overflow_count_;}

uint32_t TimerManager::getTotalCount() {return (static_cast<uint32_t>(overflow_count_) << 16) | TCNT1;}

void TimerManager::incrementOverflowCount() {overflow_count_++;}

void TimerManager::saveTimestamp() {timestamp_ = TCNT1; overflow_timestamp_ = overflow_count_;}

uint32_t TimerManager::getTotalTimestampCount() {return (static_cast<uint32_t>(overflow_timestamp_) <<
16) | timestamp_;}

ISR(TIMER1_OVF_vect) {
    TimerManager::incrementOverflowCount();
}
```

SerialHandler.h

```
/*
SerialHandler.h

Author: Noah Sergent
Date: 10/24/25

Defines a class to handle serial UART interactions of a microcontroller,
specifically made to send sensor data and receive commands to/from master PC.

Uses Timer0 on arduino UNO for timeout.
*/


#ifndef SERIALHANDLER_H
#define SERIALHANDLER_H

#include <stddef.h>
#include <stdint.h>
// #include "CircularIndex.h"
#include "VolatileIndex.h"
```

```
class SerialHandler {
  public:

    // Standard ASCII values
    static const uint8_t ENQ = 0x05;
    static const uint8_t ACK = 0x06;
    static const uint8_t STX = 0x02;
    static const uint8_t ETX = 0x03;
    static const uint8_t EOT = 0x04;

    // Message type values
    static const uint8_t MESSAGE = 'M';
    static const uint8_t DATA = 'D';
    static const uint8_t COMMAND = 'C';

    // Constants
    static const uint8_t DEFAULT_TIMEOUT_MS = 10;
    static const size_t TX_BUFFER_SIZE = 256;

    // #if TX_BUFFER_SIZE == 256
    using txindex_t = VolatileIndex;
    // #else
        // using txindex_t = CircularIndex<TX_BUFFER_SIZE>;
    // #endif

    // Flags
    static bool message_failed_;
    static bool buffer_cleared_;
    static bool buffer_overflow_;


    static void init(uint8_t ubbrl, uint8_t ubbrh);                  // Initialize UART
    static void sendString(const uint8_t *str);                      // Send a null-terminated string
    static void sendByte(const uint8_t to_send);
    static uint8_t receiveByte(void);
    static uint8_t receiveByte(uint8_t timeout_ms);
    static bool waitForByte(uint8_t expected, uint8_t timeout_ms);
    static void recieveString(uint8_t buffer[], uint8_t num_bytes);     // Receive a transmission.
    static void echoInput(uint8_t buffer_size);
    struct MsgStruct {
        uint8_t msg_type;
        uint8_t id;
        uint8_t descriptor;
        uint8_t payload_length;
        const uint8_t *payload;
```

```
    };

    static void enqueueStructuredMsg(MsgStruct *message, uint8_t full_message_length);
    static void recieveStructuredMsg(MsgStruct *message);          // Receive a structured message
    // void sendStructuredMsg(MsgStruct *message);
    static void sendByteFromBuffer(void);
    static void flushBuffer();
    // void sendMultiMsg(MsgStruct)
    static MsgStruct msgPackager(uint8_t msg_type, uint8_t id, uint8_t descriptor, uint8_t payload_length,
const uint8_t *payload_ptr);

  private:
    static volatile uint8_t tx_buffer_[TX_BUFFER_SIZE];
    static volatile txindex_t tx_head_;
    static volatile txindex_t tx_tail_;
};



#endif
```

SerialHandler.cpp

```
/*
SerialHandler.cpp

Author: Noah Sergent
Date: 10/24/25

Implements a class to handle serial UART interactions of a microcontroller,
specifically made to send sensor data and receive commands to/from master PC.

*/

#include <avr/io.h>
#include <stdint.h>
#include "SerialHandler.h"
#include "VolatileIndex.h"


//defining statics
volatile uint8_t SerialHandler::tx_buffer_[SerialHandler::TX_BUFFER_SIZE];
volatile SerialHandler::txindex_t SerialHandler::tx_head_(0);
volatile SerialHandler::txindex_t SerialHandler::tx_tail_(0);

bool SerialHandler::message_failed_ = false;
```

```
bool SerialHandler::buffer_cleared_  = true;
bool SerialHandler::buffer_overflow_ = false;

// TODO maybe use the USER LED for message failed

void SerialHandler::init(uint8_t ubbrl, uint8_t ubbrh) {
    UBRR0L = ubbrl; //Set baud rate
    UBRR0H = ubbrh;
    UCSR0B = (0 << RXCIE0) | (0 << TXCIE0) | (1 << RXEN0) | (1 << TXEN0); //  Disable RX and TX
interrupts, Enable transmitter and receiver
    UCSR0C = (0 << UMSEL01) | (0 << UMSEL00) | (0 << UPM00) | (0 << UPM01) | (0 << USBS0) | (1 <<
UCSZ01) | (1 << UCSZ00); // Asynchronous USART, No parity, 1 stop bit, 8-bit character size

    tx_head_ = 0;
    tx_tail_ = 0;

    tx_buffer_[tx_head_++] = ENQ; // Prime TX buffer
    buffer_cleared_ = true;
    message_failed_ = false;
}

void SerialHandler::sendString(const uint8_t *str){
    for (const uint8_t *p = str; *p != '\0'; ++p) {
        while (!(UCSR0A & (1 << UDRE0)));  // wait for buffer empty
        UDR0 = *p;                         // send character
    }
}

void SerialHandler::sendByte(const uint8_t to_send){ // Send a single byte //TODO implement ack handshake
    while(!(UCSR0A & (1 << UDRE0)));
    UDR0 = to_send;

    // if(to_send == ENQ) {
    //     if(!waitForByte(ACK, 0xFF)) message_failed_ = true; // TODO still need to do something with
message failed and implement timeout
    // }
}

uint8_t SerialHandler::receiveByte(void) {          // Read in a single byte
    while(!(UCSR0A & (1<<RXC0)));
    uint8_t received = UDR0;
    if(received == ENQ) sendByte(ACK);
    return received;
}
```

```
uint8_t SerialHandler::receiveByte(uint8_t timeout_ms) { // Read in a single byte with a timeout TODO:
Implement timeout
    while(!(UCSR0A & (1<<RXC0)));
    return UDR0;
}

bool SerialHandler::waitForByte(uint8_t expected, uint8_t timeout_ms) { // Await an expected byte with a
timeout TODO:Implement timeout
    while(!(UCSR0A & (1<<RXC0)));
    return UDR0 == expected;
}

void SerialHandler::recieveString(uint8_t buffer[], uint8_t num_bytes){
    uint8_t input = 0;
    uint8_t i = 0;
    while(input != '\n' && input != '\r' && i < num_bytes - 3){
        while (!(UCSR0A & (1<<RXC0)));
        input = UDR0;
        buffer[i] = input;
        i++;
    }
    buffer[i] = '\r';
    buffer[i++] = '\n';
    buffer[i++] = '\0';

}



// void SerialHandler::echoInput(uint8_t buffer_size){ //TODO remove VLA
//     uint8_t buffer[buffer_size + 3] = {0};
//     recieveString(buffer, buffer_size);
//     _delay_ms(20);
//     sendString(buffer);

// }

void SerialHandler::enqueueStructuredMsg(MsgStruct *message, uint8_t message_length) { //TODO fix VLA
    uint8_t last = (uint8_t)(tx_head_ - 1);
    if(!buffer_cleared_ && tx_head_ != tx_tail_ && tx_buffer_[last] == EOT) {
        tx_head_ = last;
    }

    if(buffer_cleared_) {
        tx_buffer_[tx_head_++] = ENQ; //If the buffer is cleared, prime the buffer again.
```

```
        buffer_cleared_ = false;
    }

    if((tx_head_ + message_length) == tx_tail_) buffer_overflow_ = true; //TODO implement buffer overflow handling

    //Unpack message into buffer
    tx_buffer_[tx_head_++] = STX;
    tx_buffer_[tx_head_++] = message->msg_type;
    tx_buffer_[tx_head_++] = message->id;
    tx_buffer_[tx_head_++] = message->descriptor;
    tx_buffer_[tx_head_++] = message->payload_length;

    uint8_t i = 0;
    while(i < message->payload_length) {
        tx_buffer_[tx_head_++] = message->payload[i];
        i++;
    }
    tx_buffer_[tx_head_++] = ETX;
    tx_buffer_[tx_head_++] = EOT;

}

void SerialHandler::sendByteFromBuffer(void) {
    if(!buffer_cleared_) sendByte(tx_buffer_[tx_tail_++]); // Send byte
    if(tx_tail_ == tx_head_) buffer_cleared_ = true; // If the tail has reached the head, then there are no more bytes to send
}

void SerialHandler::flushBuffer(void) {
    while(!buffer_cleared_) {sendByteFromBuffer();}
}

void SerialHandler::recieveStructuredMsg(MsgStruct *message){ //TODO implement without VLA
    // sendByte(ACK);
    // message->msg_type = receiveByte();
    // message->id = receiveByte();
    // message->descriptor = receiveByte();
    // message->payload_length = receiveByte();
    // uint8_t payload[message->payload_length] = {0};

}

SerialHandler::MsgStruct SerialHandler::msgPackager(uint8_t msg_type, uint8_t id, uint8_t descriptor, uint8_t payload_length, const uint8_t *payload_ptr) {
```

```
    MsgStruct message;
    message.msg_type = msg_type;
    message.id = id;
    message.descriptor = descriptor;
    message.payload_length = payload_length;
    message.payload = payload_ptr;
    return message;
}

// void SerialHandler::sendStructuredMsg(MsgStruct *message) {
//     uint8_t message_length = message->payload_length + 6;
//     enqueueStructuredMsg(message, message_length);

//     sendByte(ENQ);
//     if(waitForByte(ACK, DEFAULT_TIMEOUT_MS)) {
//         for(uint8_t i = 0; i < message_length; i++) {
//             // sendByte(to_send[i]);
//         }
//         sendByte(EOT);
//     } else {
//         message_failed_ = true;
//     }
// }
```

IRSensor.h

```
/*
IRSensor.h

Author: Noah Sergent
Date: 10/28/25

Defines a class to handle the beam break sensor input. Includes functions to
measure the duration of a beam break, and the time of a beam break.
*/


#ifndef IRSENSOR_H
#define IRSENSOR_H

#include "TimerManager.h"
#include <avr/io.h>
#include <stdint.h>

# define DEBOUNCE_TICKS 4000
```

```
enum class SensorPosition : uint8_t {First, Last, Other};

enum class BeamStatus : uint8_t {Broken, Intact};


class IRSensor : public TimerManager {
    public:
        // static volatile bool send_finished_msg_flag;
        static uint8_t pcmsk0;
        const uint8_t sensor_id_;
        volatile uint8_t& port_;
        uint8_t pin_location_;
        SensorPosition position_;
        BeamStatus beamstatus_;
        volatile uint32_t break_duration_;
        volatile uint32_t break_start_time_;
        volatile uint32_t last_transition_time_;
        volatile bool start_time_ready_to_send_;
        volatile bool duration_ready_to_send_;

        // Constants
        // Size (in bytes) of the data fields recorded
        static const uint8_t RECORD_SIZE = 4;

        // Descriptors for message
        static const uint8_t DURATION_DESCRIPTOR = 'D';
        static const uint8_t START_TIME_DESCRIPTOR = 'T';

        IRSensor(uint8_t id, volatile uint8_t &port, uint8_t pin_location,
SensorPosition pos);
        IRSensor(uint8_t id, volatile uint8_t &port, uint8_t pin_location);

        static void configureInterrupts(uint8_t pcmsk0);

        bool wasTriggered(uint8_t saved_port);
        void recordBreakEvent();
        uint32_t getDuration();
        uint32_t getStartTime();

    private:
        uint8_t getSensorVal();                                  // Returns the
value of the pin moved to the LSB
        uint8_t getSensorVal(uint8_t saved_port);
        void updateBeamStatus();
```

```
        void updateBeamStatus(uint8_t saved_port);

};



#endif
```

IRSensor.cpp

```
/*
IRSensor.cpp

Author: Noah Sergent
Date: 10/28/25

Implements a class to handle the beam break sensor input. Includes functions to
measure the duration of a beam break, and the time of a beam break.
*/

#include "IRSensor.h"
#include <avr/io.h>
#include <stdint.h>

IRSensor::IRSensor(uint8_t id, volatile uint8_t &port, uint8_t pin_location,
SensorPosition pos):
    sensor_id_(id),
    port_(port),
    pin_location_(pin_location),
    position_(pos)
    {
    beamstatus_ = BeamStatus::Intact;
    break_duration_ = 0;
    break_start_time_ = 0;
    start_time_ready_to_send_ = false;
    duration_ready_to_send_ = false;
    // send_finished_msg_flag = false;
    last_transition_time_ = 0;
}

IRSensor::IRSensor(uint8_t id, volatile uint8_t &port, uint8_t pin_location):
sensor_id_(id),
port_(port),
pin_location_(pin_location),
position_(SensorPosition::Other)
```

```
{
    beamstatus_ = BeamStatus::Intact;
    break_duration_ = 0;
    break_start_time_ = 0;
    start_time_ready_to_send_ = false;
    duration_ready_to_send_ = false;
    // send_finished_msg_flag = false;
    last_transition_time_ = 0;
}

void IRSensor::configureInterrupts(uint8_t pcmsk0) { // TODO implement interrupt
masking for other ports
    DDRB &= ~(pcmsk0);   // ensure pins are inputs
    PORTB |= pcmsk0;     // enable internal pull-ups
    PCMSK0 = pcmsk0;
    // PCMSK1 = pcmsk1;
    // PCMSK2 = pcmsk2;

    PCICR  |= (1 << PCIE0);                    // enable PCINT[7:0] group
}

uint8_t IRSensor::getSensorVal() {
    return (port_ >> pin_location_) & 0x01;
}

uint8_t IRSensor::getSensorVal(uint8_t saved_port) {
    return (saved_port >> pin_location_) & 0x01;
}

bool IRSensor::wasTriggered(uint8_t saved_port) {
    BeamStatus previous = beamstatus_;
    updateBeamStatus(saved_port);

    if (beamstatus_ == previous) return false;

    // Debounce
    uint32_t now = TimerManager::getTotalTimestampCount();
    uint32_t delta = now - last_transition_time_;

    if (last_transition_time_ != 0 && delta < DEBOUNCE_TICKS) {
        // If the time passed since last activation is less than specified,
revert status
        beamstatus_ = previous;
        return false;
    }
```

```
    last_transition_time_ = now;
    return true;
}

void IRSensor::updateBeamStatus() {
    if(getSensorVal() == 0) beamstatus_ = BeamStatus::Broken;
    else beamstatus_ = BeamStatus::Intact;
}

void IRSensor::updateBeamStatus(uint8_t saved_port) {
    if(getSensorVal(saved_port) == 0) beamstatus_ = BeamStatus::Broken;
    else beamstatus_ = BeamStatus::Intact;
}

void IRSensor::recordBreakEvent() {
    if(beamstatus_ == BeamStatus::Broken) {
        if((position_ == SensorPosition::First)) {
            startTimer1(true);
            saveTimestamp();
            // send_finished_msg_flag = false;
        }
        break_start_time_ = getTotalTimestampCount();
        start_time_ready_to_send_ = true;
    }
    if(beamstatus_ == BeamStatus::Intact) {
        break_duration_ = getTotalTimestampCount() - break_start_time_;
        if(position_ == SensorPosition::Last) {
            stopTimer1();
            // send_finished_msg_flag = true;
        }
        duration_ready_to_send_ = true;
    }
}

uint32_t IRSensor::getDuration() {return break_duration_;}
uint32_t IRSensor::getStartTime() {return break_start_time_;}
```

VolatileIndex.h

```
/*
Volatile.h

Author: Noah Sergent
```

```
Date: 10/24/25

This class specifies a volatile uint_8 that supports the ++ postfix and += operators
*/

#ifndef VOLATILEINDEX_H
#define VOLATILEINDEX_H
#include <stdint.h>

class VolatileIndex {
   public:
      volatile uint8_t value;

      VolatileIndex() : value(0){};
      VolatileIndex(uint8_t init) : value(init){};

      inline volatile VolatileIndex& operator++() volatile {
         value = value + 1;
         return *this;
      }

      inline VolatileIndex operator++(int) volatile {
         VolatileIndex temp = *this;
         this->operator++();
         return temp;
      }

      inline volatile VolatileIndex& operator=(uint8_t newval) volatile {
         value = newval;
         return *this;
      }

      inline operator uint8_t() const volatile {return value;}


};


#endif
```

Host PC (Python):

MLM

```python
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import warnings
warnings.filterwarnings('ignore')

# Load data
df = pd.read_csv('Log_11-14-25_15-41-06.csv')

# Create feature matrix: For each sensor reading, we want to predict the NEXT big T
# Features: current big T, current little t
# Target: next big T

# We'll create training data by pairing current sensor with next sensor
training_data = []
for i in range(len(df) - 1):
    current_row = df.iloc[i]
    next_row = df.iloc[i + 1]

    current_sensor = current_row['Sensor ID']
    next_sensor = next_row['Sensor ID']

    # Only create training pair if we're going to the next sequential sensor
    # Skip when we jump back to sensor 0 (new run)
    if next_sensor == current_sensor + 1:
        training_data.append({
            'current_sensor': current_sensor,
            'current_big_T': current_row['T'],
            'current_little_t': current_row['t'],
            'next_sensor': next_sensor,
            'next_big_T': next_row['T']
        })

training_df = pd.DataFrame(training_data)

# Build separate models for each sensor transition
models = {}
results = {}

print("="*80)
print("BUILDING MULTIPLE LINEAR REGRESSION MODELS")
print("="*80)
```

```
data_0_to_1 = training_df[training_df['current_sensor'] == 0]

# Single feature
X1 = data_0_to_1[['current_little_t']].values
y1 = data_0_to_1['next_big_T'].values

X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42
)

model1 = LinearRegression()
model1.fit(X1_train, y1_train)

y1_pred = model1.predict(X1_test)

r2_1 = r2_score(y1_test, y1_pred)
rmse_1 = np.sqrt(mean_squared_error(y1_test, y1_pred))
mae_1 = mean_absolute_error(y1_test, y1_pred)

print(f"Training samples: {len(X1_train)}")
print(f"Test samples: {len(X1_test)}")
print(f"\nFeature: current_little_t")
print(f"Intercept: {model1.intercept_:.6f}")
print(f"Coefficient: {model1.coef_[0]:.6f}")
print(
    f"Equation: next_big_T = {model1.intercept_:.6f} + "
    f"{model1.coef_[0]:.6f} * current_little_t"
)
print(f"\nPerformance Metrics:")
print(f"  R² Score: {r2_1:.4f}")
print(f"  RMSE: {rmse_1:.6f} seconds")
print(f"  MAE: {mae_1:.6f} seconds")


# Model 2: Predict big T at Sensor 2 from Sensor 1
print("\n[MODEL 2] Predict big T at Sensor 2 (from Sensor 1)")
print("-" * 80)

data_1_to_2 = training_df[training_df['current_sensor'] == 1]
X2 = data_1_to_2[['current_big_T', 'current_little_t']].values
y2 = data_1_to_2['next_big_T'].values

X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2, random_state=42)

model2 = LinearRegression()
```

```
model2.fit(X2_train, y2_train)
y2_pred = model2.predict(X2_test)

r2_2 = r2_score(y2_test, y2_pred)
rmse_2 = np.sqrt(mean_squared_error(y2_test, y2_pred))
mae_2 = mean_absolute_error(y2_test, y2_pred)

print(f"Training samples: {len(X2_train)}")
print(f"Test samples: {len(X2_test)}")
print(f"\nFeatures: current_big_T, current_little_t")
print(f"Intercept: {model2.intercept_:.6f}")
print(f"Coefficients: {model2.coef_}")
print(f"Equation: next_big_T = {model2.intercept_:.6f} + {model2.coef_[0]:.6f} * current_big_T +
{model2.coef_[1]:.6f} * current_little_t")
print(f"\nPerformance Metrics:")
print(f"  R² Score: {r2_2:.4f}")
print(f"  RMSE: {rmse_2:.6f} seconds")
print(f"  MAE: {mae_2:.6f} seconds")

models['sensor_1_to_2'] = model2
results['sensor_1_to_2'] = {'r2': r2_2, 'rmse': rmse_2, 'mae': mae_2}

# Model 3: Predict big T at Sensor 3 from Sensor 2
print("\n[MODEL 3] Predict big T at Sensor 3 (from Sensor 2)")
print("-" * 80)

data_2_to_3 = training_df[training_df['current_sensor'] == 2]
X3 = data_2_to_3[['current_big_T', 'current_little_t']].values
y3 = data_2_to_3['next_big_T'].values

X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, test_size=0.2, random_state=42)

model3 = LinearRegression()
model3.fit(X3_train, y3_train)
y3_pred = model3.predict(X3_test)

r2_3 = r2_score(y3_test, y3_pred)
rmse_3 = np.sqrt(mean_squared_error(y3_test, y3_pred))
mae_3 = mean_absolute_error(y3_test, y3_pred)

print(f"Training samples: {len(X3_train)}")
print(f"Test samples: {len(X3_test)}")
print(f"\nFeatures: current_big_T, current_little_t")
print(f"Intercept: {model3.intercept_:.6f}")
print(f"Coefficients: {model3.coef_}")
```

```
print(f"Equation: next_big_T = {model3.intercept_:.6f} + {model3.coef_[0]:.6f} * current_big_T +
{model3.coef_[1]:.6f} * current_little_t")
print(f"\nPerformance Metrics:")
print(f"  R² Score: {r2_3:.4f}")
print(f"  RMSE: {rmse_3:.6f} seconds")
print(f"  MAE: {mae_3:.6f} seconds")

models['sensor_2_to_3'] = model3
results['sensor_2_to_3'] = {'r2': r2_3, 'rmse': rmse_3, 'mae': mae_3}
```

main.py

```
# main.py
# Main entry point for the program
# Author: Noah Sergent

import SerialHandler
import GUI
import Logger
from SerialHandler import SerialHandler as sh
import threading
from PySide6.QtWidgets import QApplication
import sys
import Error
import ModuleIntegration
from ModuleIntegration import Coordinator
import Prediction
import CONST
from MessageProcessor import MessageProcessor
from PredictionPlanner import PredictionPlanner
from queue import Queue
from Animator import Animator
from ConfigManager import ConfigManager
import argparse

if __name__ == "__main__":
    app = QApplication(sys.argv)

    # --- 1) Startup mode selection ---
    startup_window = GUI.StartupDialog()
    startup_window.show_window()
    mode = startup_window.selected_mode
```

```
    confman = ConfigManager()
    logger = Logger.Logger(confman)
    global_message_queue = Queue()

    # --- 3) Create serial handlers based on mode ---
    daq = None
    led = None

    if mode in (CONST.DAQ_ONLY, CONST.FULL, CONST.LATENCY_TEST):
        daq = sh(CONST.DAQ, CONST.DAQ_COM, CONST.DAQ_BAUD, logger=logger,
global_msg_queue=global_message_queue)
        animator = None

    if mode in (CONST.LED_ONLY, CONST.FULL):
        led = sh(CONST.LED_CONTROLLER, CONST.LED_DRIVER_COM,
CONST.LED_DRIVER_BAUD, logger=logger, global_msg_queue=global_message_queue)
        animator = Animator(CONST.REFRESH, led_sh=led)

    if mode == CONST.LATENCY_TEST:
        led = sh(CONST.LED_CONTROLLER, CONST.LED_DRIVER_COM,
CONST.LED_DRIVER_BAUD, logger=logger, global_msg_queue=global_message_queue)
        animator = Animator(CONST.REFRESH, led_sh=led)

    if mode != CONST.SFP_DEBUG:
        prediction_planner = PredictionPlanner(logger=logger)

        if mode in (CONST.LED_ONLY, CONST.FULL, CONST.LATENCY_TEST):
            message_processor = MessageProcessor(
                prediction_planner,
                logger,
                led,
                animator=animator,
                config=confman,
                latency_test=(mode == CONST.LATENCY_TEST)
            )
        else:
            message_processor = MessageProcessor(
                prediction_planner,
                logger,
                led,
                animator=None,
                config=confman
            )

    # --- Coordinator ---
```

```
    coord = ModuleIntegration.Coordinator(
        daq_sh=daq,
        led_sh=led,
        logger=logger,
        conf=None,
        global_message_queue=global_message_queue,
        message_processor=message_processor,
        animator=animator
    )
    Error.set_recipients(logger, coord)

    # --- Main window ---
    main_window = GUI.MainWindow(
        log_path=logger.get_log_path(),
        dlog_path=logger.get_dlog_path(),
        mode=mode
    )

    # --- Wire coordinator <-> GUI ---
    ModuleIntegration.connect_coordinator_to_gui(coord, main_window)

    # --- Show + start serial threads ---
    main_window.showMaximized()
    coord.start_serial_threads()
    if mode != CONST.SFP_DEBUG:
        prediction_planner.start()
    if mode in (CONST.LED_ONLY, CONST.FULL, CONST.LATENCY_TEST):
        animator.start()

    sys.exit(app.exec())
```

SerialHandler.py

```
# SerialHandler.py
# Author: Noah Sergent

import os
import serial
import time
from pprint import pprint
from queue import Queue
import math
import csv
```

```
import threading

import Logger
import ConfigManager
import Error
import GenFunct
import CONST

current_directory = os.path.dirname(__file__)

class SerialHandler(serial.Serial):

    # Expected incoming message structure:
    # ASCII | ASCII |   ASCII   | HEX |   ASCII    |     HEX(n)     |
HEX(n_length) | ASCII | ASCII
    #  ENQ   |  STX  | Msg Type | ID  | Descriptor | Payload Length
|    Payload    |  ETX  |  EOT

    # csv_header1 = ["Sensor_ID", "T", "t"]

    def __init__(self,  attached_device, port_name="COM5", baud_rate=9600,
global_msg_queue: Queue[dict] | None = None, verbose=False, timeout=None, logger:
Logger.Logger | None = None):
        self.failed = False

        try:
            super().__init__(port=port_name, baudrate=baud_rate,
timeout=timeout)   # Open COM port
        except (FileNotFoundError, serial.SerialException) as e:
            Error.report(f"Couldn't open {port_name}: {e}", "SerialHandler init")
            self.failed = True

        if logger is None:
            self.logger = Logger.Logger()
        else:
            self.logger = logger



        if not self.failed:

            if verbose:
               Error.event(f"Port {port_name} opened at {baud_rate} baud\n",
"SerialHandler init")
```

```
            time.sleep(0.0025)

        self.source = attached_device

        self._raw_queue: Queue[int] = Queue()
        if global_msg_queue is None:
            self._msg_queue: Queue[dict] = Queue()
        else:
            self._msg_queue = global_msg_queue

        self._reader_thread = threading.Thread ( target=self._reader_loop,
daemon=True, name="ReaderLoop")
        self._parser_thread = threading.Thread ( target=self._parser_loop,
daemon=True, name="ParserLoop")

    def set_logger(self, logger: Logger.Logger):
        self.logger = logger

    def start(self):
        self._reader_thread.start()
        self._parser_thread.start()

    def _reader_loop(self):
        """Drains uart, should be run in a thread
        """
        while True:
            try:
                data = self.read(1)
                if data:
                    self._raw_queue.put(data[0])
                    self.logger.dlog(f"{data[0]} | ", end_char="",
add_timestamp=False)
            except Exception as e:
                Error.report(f"Read error: {e}", "Serial Reader")

    def _parser_loop(self):
        E_SOURCE = "SerialHandler parser"
        while True:
            try:
                while True:
                    byte = self._raw_queue.get()
                    # Wait for frame start
                    if (byte == CONST.ENQ):
                        self.logger.dlog("New Transmission")
                        break
```

```
                while True:
                    # parse message into dict
                    byte = self._raw_queue.get()

                    if (byte == CONST.EOT): # Transmission over
                        break
                    elif(byte != CONST.STX): # If byte after EOT != STX, then
desync occured
                        Error.warn("Serial frame desync, waiting for next frame",
E_SOURCE)
                        continue
                    msg = {}
                    msg["Message Type"] = chr(self._raw_queue.get())
                    msg["ID"] = self._raw_queue.get()
                    msg["Descriptor"] = chr(self._raw_queue.get())
                    msg["Payload Length"] = self._raw_queue.get()
                    msg["Payload"] = [self._raw_queue.get() for _ in
range(msg["Payload Length"])]
                    msg["Parsed Timestamp"] = GenFunct.get_timestamp(True)
                    msg["Source"] = self.source

                    byte = self._raw_queue.get()
                    if byte != CONST.ETX:
                        Error.report(f"Expected ETX, got {byte}, dropping
message: {msg}", E_SOURCE)
                    else:
                        self._msg_queue.put(msg)
                        self.logger.dlog(f"Message added to queue {msg}",
start_char="\n")

            except Exception as e:
                Error.report(f"Parser error: {e}", E_SOURCE)


    def send_frame(self, msg_type, device_id, descriptor, payload, ignore_length:
bool = False):
        E_SOURCE = "Send Frame"
        try:
            frame = bytes([
                CONST.ENQ,
                CONST.STX,
                msg_type, device_id, descriptor,
                len(payload) if not ignore_length else 0xff,
                *payload,
```

```
                CONST.ETX,
                CONST.EOT
            ])
            self.write(frame)
            # self.logger.dlog(f"Sent frame: {frame} \n to {self.source}")
        except Exception as e:
            Error.report(f"{e}", E_SOURCE)


    def to_seconds(self, x):
        return (int.from_bytes(x, byteorder="big"))/2e6


# if __name__ == "__main__":
#     serial_handler = SerialHandler(verbose=True,port_name="COM8")
#     while(True):
#         serial_handler.receive_transmission(print_log=True)
        # # serial_handler.print_all_incoming()
        # printed_messages = []

        # for message in serial_handler.incoming_messages:
        #     if message not in printed_messages:
        #         printed_messages.append(message)
        #         pprint(message)
        #         decimal_payload = int.from_bytes(message["Payload"],
byteorder="big")
        #         # print(f"number messages
{len(serial_handler.incoming_messages)}")
        #         print(f"Decimal Payload: {decimal_payload}")
        #         print(f"Seconds: {decimal_payload/2e6}")
```

MessageProcessor.py

```
# MessageProcessor.py
# Author: Noah Sergent

from queue import Queue
import GenFunct
import CONST
import Error
import threading
import time
```

```
from PredictionPlanner import PredictionPlanner
import Logger
from SerialHandler import SerialHandler
import Animator
from ConfigManager import ConfigManager
from Run import Run

class MessageProcessor():
    """Handles all processing of incoming and outgoing data
    Reads from incoming parsed message queue populated by SerialHanlder
    Directly sends frames through SerialHandler
    """

    RUN_TIMEOUT_S = 10.0
    RUN_COOLDOWN_S = 2.0

    def __init__(self, prediction_planner: PredictionPlanner | None = None,
logger: Logger.Logger | None = Logger.Logger(), led_sh: SerialHandler | None =
None, animator: Animator.Animator | None = None, config: ConfigManager | None =
None, latency_test: bool = False):

        self._latency_test = latency_test
        self._latency_samples = []
        self._current_receive_time = None

        self.outgoing = Queue()

        self.logger = logger
        self.prediction_planner = prediction_planner
        self.led_sh = led_sh
        self.led_times_queue = Queue()
        self.animator = animator
        self.confman = config

        # DAQ Data Vars
        self.log_rows = Queue()
        self.run_id = 0
        self.current_run: Run | None = None
        self._last_sensor_id = self.confman.get_last_sensor_id()
        self._run_ended_at = 0.0
        self._pending_timers: list[threading.Timer] = []

        # Dictionary to route incoming messages to the correct handlers
        self._routing = {
            (CONST.DAQ, CONST.DATA) : self._handle_daq_data,
```

```
            (CONST.ANY, CONST.ASCII): self._handle_ascii_message,
            (CONST.LED_CONTROLLER, CONST.STATUS) : self._handle_led_status
        }

        self._inbound_thread = threading.Thread(target=self._inbound_worker,
daemon=True, name="InboundWorker")
        self._outbound_thread = threading.Thread(target=self._outbound_worker,
daemon=True, name="OutboundWorker")
        self._watchdog_thread = threading.Thread(target=self._watchdog_worker,
daemon=True, name="RunWatchdog")

    def set_incoming_msg_queue(self, incoming_msg_queue: Queue):
        """Sets the queue MessageProcessor will pull from

        Args:
            incoming_msg_queue (Queue[dict]): Queue of parsed messages
        """
        self.incoming = incoming_msg_queue

    def start(self):
        """Starts inbound, outbound, and watchdog worker threads
        """
        self._inbound_thread.start()
        self._outbound_thread.start()
        self._watchdog_thread.start()

    def _inbound_worker(self):
        """Processes inbound parsed messages from message queue
        """
        E_SOURCE = "MessageProcessor inbound worker"
        while (True):
            try:
                msg = self.incoming.get()
                msg_type = msg["Message Type"]
                source = msg["Source"]

                if msg_type == CONST.ASCII:
                    source = CONST.ANY

                handler = self._routing.get((source, msg_type))

                if handler:
                    handler(msg)
                else:
                    Error.report(f"No handler for message msg={msg}", E_SOURCE)
```

```
            except Exception as e:
                Error.report(f"Exception: {e}", E_SOURCE)

    def _outbound_worker(self):
        """Processes and queues commands to LED Controller
        """
        self.prediction_planner.set_times_queue(self.led_times_queue)
        while True:
            intermediate_times_dict = self.led_times_queue.get()
            if not intermediate_times_dict:
                continue
            for key, value in intermediate_times_dict.items():
                timer = threading.Timer(value, self.set_led_in_buffer,
args=[key])
                self._pending_timers.append(timer)
                timer.start()

    def _watchdog_worker(self):
        """Periodically checks for stale runs and resets them
        """
        E_SOURCE = "RunWatchdog"
        while True:
            time.sleep(1.0)
            if self.current_run and
self.current_run.is_stale(self.RUN_TIMEOUT_S):
                Error.warn(f"Run {self.current_run.run_id} timed out, resetting",
E_SOURCE)
                self._end_run()

    def _handle_daq_data(self, msg):
        """Handles data incoming from DAQ,
           Pairs duration and start time values for a sensor,
           submits data to PredictionPlanner and log

        Args:
            msg (dict): Parsed incoming data message from DAQ
        """
        E_SOURCE = "MessageProcessor DAQ Data Handling"


        if self._latency_test:
            self._current_receive_time = time.perf_counter()


        # Start a new run when sensor 0 fires and no run is active
```

```
        if msg["ID"] == 0 and (self.current_run is None or
self.current_run.is_complete()):
            if not self._start_new_run():
                return

        if self.current_run is None:
            Error.warn("Data received outside of a run, discarding", E_SOURCE)
            return

        ready, data, error = self.current_run.accept(msg)

        if error:
            Error.warn(error, E_SOURCE)

        if not ready:
            return

        # if data["Duration"] < CONST.MINIMUM_DURATION:
        #     Error.warn("Duration < minimum, likely false reading, dropping
data", E_SOURCE)
        #     return

        if self.current_run.is_complete():
            Error.event("Run Done!", E_SOURCE)
            self.logger.log([data["Run ID"], data["Event ID"], data["Timestamp"],
data["Sensor ID"], data["Duration"]])
            self._end_run()
            return

        self._submit_data(data)
        if not CONST.INCLUDE_PREDICTION_IN_LOG:
            self.logger.log([data["Run ID"], data["Event ID"], data["Timestamp"],
data["Sensor ID"], data["Duration"]])

    def _start_new_run(self) -> bool:
        """Create a fresh Run instance and increment run id.
        Respects a cooldown period after the previous run ended
        to prevent false triggers from the ball bouncing.

        Returns:
            bool: True if a new run was started, False if still in cooldown
        """
        if time.time() - self._run_ended_at < self.RUN_COOLDOWN_S:
            Error.warn(f"Run cooldown active, ignoring trigger ({time.time() -
self._run_ended_at:.2f}s since last run)", "MessageProcessor")
```

```
            return False

        self.run_id += 1
        self.current_run = Run(self.run_id, self._last_sensor_id)
        Error.event(f"Run {self.run_id} started", "MessageProcessor")
        return True

    def _end_run(self):
        run_id = self.current_run.run_id if self.current_run else "?"
        time.sleep(0.5)
        for timer in self._pending_timers:
            timer.cancel()
        self._pending_timers.clear()

        if self.animator:
            self.animator.all_off()

        if self._latency_test and self._latency_samples:
            max_latency_ms = max(self._latency_samples) * 1000
            avg_latency_ms = sum(self._latency_samples) /
len(self._latency_samples) * 1000
            min_latency_ms = min(self._latency_samples) * 1000
            result_text = (
                f"Run {run_id} Latency Results:\n"
                f"  Maximum Latency: {max_latency_ms:.2f} ms\n"
                f"  Average Latency: {avg_latency_ms:.2f} ms\n"
                f"  Minimum Latency: {min_latency_ms:.2f} ms\n"
                f"  Samples: {len(self._latency_samples)}"
            )
            Error.event(result_text, "Latency Test")
            self._latency_samples.clear()
            self._current_receive_time = None

        self.current_run = None
        self._run_ended_at = time.time()
        Error.event(f"Run {run_id} ended, cooldown started", "MessageProcessor")

    def _handle_ascii_message(self, msg):
        source = msg["Source"]
        Error.event(f"Messsage received: {"".join(chr(b) for b in
msg["Payload"])}", source)

    def _handle_led_status(self, msg):
        pass
```

```
    def _submit_data(self, data):
        """Adds sensor data to attached prediction planner queue

        Args:
            data (dict): Dictionary containing start time and duration of ir
sensor activation
        """
        self.prediction_planner.incoming_data.put(data)

    def all_led_on(self):
        self.led_sh.send_frame(
            ord('C'),
            0xFF,
            ord(CONST.ALL_LED_ON_DESCRIPTOR),
            [0]
        )

    def all_led_off(self):
        self.led_sh.send_frame(
            ord('C'),
            0xFF,
            ord(CONST.ALL_LED_OFF_DESCRIPTOR),
            [0]
        )

    def set_led_in_buffer(self, index):
        self.animator.set_led(index)
        if self._latency_test and self._current_receive_time is not None:
            latency = time.perf_counter() - self._current_receive_time
            self._latency_samples.append(latency)
```

Run.py

```
# Run.py
# Author: Noah Sergent

import time
import CONST
import Error
import GenFunct

class Run:
    """Represents a single ball run through the track.
    Manages sensor data pairing, ordering, and validation for one run lifecycle.
    """
```

```
    E_SOURCE = "Run"
    DEBOUNCE_S = 0.05

    def __init__(self, run_id: int, last_sensor_id: int):

        # Identity
        self.run_id = run_id
        self.event_id = 0
        self.last_sensor_id = last_sensor_id

        # Sensor tracking
        self.max_sensor_id_seen = -1
        self._last_sensor_times: dict[int, float] = {}

        # Pairing state
        self.partial: dict = {
            "Sensor ID": None,
            "Run ID": run_id,
            "Event ID": None,
            "Timestamp": None,
            "Duration": None
        }

        # Timing
        self._created_at = time.time()
        self._last_activity_at = time.time()

    # ------------------------------------------------------------------
    # Public interface (called by MessageProcessor)
    # ------------------------------------------------------------------

    def accept(self, msg: dict) -> tuple[bool, dict | None, str | None]:
        """Main entry point for incoming sensor data.
        Runs all validation, then attempts to pair the data.

        Args:
            msg: Parsed message dict from SerialHandler

        Returns:
            (ready, completed_partial, error_message)
            - ready: True if a complete sensor pair is available
            - completed_partial: the finished data dict, or None
            - error_message: description of any problem, or None
        """
        self._last_activity_at = time.time()
```

```
        error = self._validate(msg)
        if error:
            return False, None, error

        ready = self._pair(msg)

        if ready:
            completed = self._finalize_partial()
            return True, completed, None

        return False, None, None

    def is_stale(self, timeout_s: float) -> bool:
        """Called by MessageProcessor watchdog to detect dead/stuck runs.

        Args:
            timeout_s: Seconds of inactivity before run is considered stale
        """
        return time.time() - self._last_activity_at > timeout_s


    # ------------------------------------------------------------------
    # Validation (all checks before pairing is attempted)
    # ------------------------------------------------------------------

    def _validate(self, msg: dict) -> str | None:
        """Runs all validation checks on incoming message.
        Returns an error string if invalid, None if ok.
        """
        error = self._check_debounce(msg)
        if error: return error

        error = self._check_ordering(msg)
        if error: return error

        return None

    def _check_debounce(self, msg: dict) -> str | None:
        # Only debounce start-time messages — duration messages are always
        # the second half of the same activation and must never be suppressed
        if msg["Descriptor"] != CONST.START_TIME_DESCRIPTOR:
            return None

        sid = msg["ID"]
```

```
        last = self._last_sensor_times.get(sid, 0)
        if time.time() - last < self.DEBOUNCE_S:
            return f"Debounced sensor {sid}"
        self._last_sensor_times[sid] = time.time()
        return None

    def _check_ordering(self, msg: dict) -> str | None:
        """Rejects stale data from sensors behind the ball's current position."""
        if msg["ID"] < self.max_sensor_id_seen:
            return f"Stale data from sensor {msg['ID']}, already at
{self.max_sensor_id_seen}"
        self.max_sensor_id_seen = max(self.max_sensor_id_seen, msg["ID"])
        return None

    # ------------------------------------------------------------------
    # Pairing
    # ------------------------------------------------------------------

    def _pair(self, msg: dict) -> bool:
        sid = msg["ID"]
        descriptor = msg["Descriptor"]

        if descriptor == CONST.START_TIME_DESCRIPTOR:
            self._reset_partial(sid)
            self.partial["Timestamp"] = GenFunct.to_seconds(msg["Payload"])
            return False

        elif descriptor == CONST.DURATION_DESCRIPTOR:
            if self.partial["Sensor ID"] != sid or self.partial["Timestamp"] is
None:
                self._reset_partial()
                return False
            self.partial["Duration"] = GenFunct.to_seconds(msg["Payload"])
            return True

        return False

    def _reset_partial(self, sensor_id: int | None = None):
        """Clears partial dict, optionally seeding with a new sensor id."""
        self.partial = dict.fromkeys(self.partial, None)
        self.partial["Run ID"] = self.run_id
        self.partial["Sensor ID"] = sensor_id

    # ------------------------------------------------------------------
    # Finalization
```

```
    # ---------------------------------------------------------------------

    def _finalize_partial(self) -> dict:
        """Stamps event id onto completed partial, increments counter, returns
copy."""
        self.partial["Event ID"] = self.event_id
        self.event_id += 1
        completed = self.partial.copy()
        self._reset_partial()
        return completed

    def is_complete(self) -> bool:
        return self.max_sensor_id_seen >= self.last_sensor_id
```

PredictionPlanner.py

```
# PredictionPlanner.py
# Author: Noah Sergent

import ConfigManager
import Prediction
from queue import Queue
import threading
import Error
from Logger import Logger
import CONST

class PredictionPlanner():
    def __init__(self, logger: Logger | None = None):
        self._model_data = Prediction.load_model()
        self.incoming_data = Queue()
        self._worker_thread = threading.Thread ( target=self._worker,
daemon=True, name="PredictionPlanner_Worker")
        self.logger = logger
        self.current_ball_distance = None

    def start(self):
        self._worker_thread.start()
        Error.event("PredictionPlanner thread started", "PredictionPlanner
start")

    def set_times_queue(self, target_queue):
        self.times_queue = target_queue

    def _worker(self):
        E_SOURCE = "PredictionPlanner worker"
```

```
        while True:
            try:
                data_row = self.incoming_data.get()
                prediction = self._get_prediction(data_row)
                prediction = prediction[0]
                if prediction == 0:
                    prediction = "N/A"
                else:
                    intermediate_times = self._get_intermediate_times(prediction,
data_row)
                self._submit_schedule(intermediate_times)

                if self.logger and CONST.INCLUDE_PREDICTION_IN_LOG:
                    self.logger.log([
                    data_row["Run ID"],
                    data_row["Event ID"],
                    data_row["Timestamp"],
                    data_row["Sensor ID"],
                    data_row["Duration"],
                    prediction          # prediction is a numpy array, [0] gets
the scalar
                    ])
            except Exception as e:
                Error.report(f"{e}", E_SOURCE)


    def _get_prediction(self, data_row):
        return Prediction.get_prediction(self._model_data, data_row["Timestamp"],
data_row["Duration"], data_row["Sensor ID"])

    def _get_intermediate_times(self, prediction, data_row):
        E_SOURCE = "PredictionPlanner Schedule Generation"

        ID = data_row["Sensor ID"]
        if ID == 4:
            return None

        this_sensor_distance = CONST.SENSOR_DISTANCES[ID]
        next_sensor_distance = CONST.SENSOR_DISTANCES[ID + 1]
        this_sensor_time = data_row["Timestamp"]
        next_sensor_time = prediction

        schedule = {}
```

```
        s = (next_sensor_distance - this_sensor_distance) / (next_sensor_time -
this_sensor_time)

        for led_index, led_distance in CONST.LED_DISTANCES.items():
            if led_distance < this_sensor_distance or led_distance >
next_sensor_distance:
                continue

            time = (led_distance - this_sensor_distance) / s

            schedule[led_index] = time + CONST.SCHEDULE_OFFSET_S

        self.logger.dlog(f"Schedule Generated: {schedule}", E_SOURCE)

        return schedule

    def _update_distance(self):
        pass









    def _submit_schedule(self, times_dict):
        self.times_queue.put(times_dict)
```

Prediction.py

```
# Prediction.py
# Used to interface with the MLM
# Author: Noah Sergent

import time
import pickle
from GenFunct import get_timestamp
from pprint import pprint
import CONST

# example times
T1 = 5e-7
D1 = 0.079605

T2 = 0.473603
D2 = 0.0404265
```

```
T3 = 0.9790895
D3 = 0.0272965

T4 = 1.275144
D4 = 0.0100975

sensor_stretches = {0 :"sensor_0_to_1", 1: "sensor_1_to_2", 2: "sensor_2_to_3",
3: "sensor_3_to_4"}

def load_model():
    picklefile_path = 'python/models.pkl' # Replace with your actual file path#
Open the file in binary read mode ('rb')with open(file_path, 'rb') as file:

    with open(picklefile_path, 'rb') as file:
        data = pickle.load(file)

    return data

def get_prediction(data, timestamp, duration, sensor):
    if sensor == 0:
        prediction = data[CONST.SENSOR_STRETCHES[sensor]].predict([[duration]])
    elif sensor == 4:
        prediction = [0]
    else:
        prediction = data[CONST.SENSOR_STRETCHES[sensor]].predict([[timestamp,
duration]])
    return prediction

def get_prediction_time():
    timestamp1 = time.perf_counter()
    print(get_prediction(T1, D1, 0))
    timestamp2 = time.perf_counter()
    print(get_prediction(T2, D2, 0))
    timestamp3 = time.perf_counter()
    print(get_prediction(T3, D3, 0))
    timestamp4 = time.perf_counter()
    print(get_prediction(T4, D4, 0))
    timestamp5 = time.perf_counter()

    return timestamp1, timestamp2, timestamp3, timestamp4, timestamp5

if __name__ == "__main__":
    load_model()
    pprint(get_prediction_time())
```

```
    # print(data[sensor_stretches[0]].n_features_in_)
```

GUI.py

```
# GUI.py
# Creates GUI SFP
# Author: Noah Sergent

import sys
import csv

from PySide6.QtCore import Qt, QAbstractTableModel, QTimer, QEventLoop, QRectF
from PySide6.QtWidgets import (
    QApplication, QMainWindow, QTableView, QPlainTextEdit,
    QWidget, QHBoxLayout, QPushButton, QVBoxLayout, QSpinBox, QLabel, QComboBox
)
from PySide6.QtGui import QPainter, QColor
import CONST


MODE = None


class CSVWatcher:
    def __init__(self, file_path=None):
        self.file_path = None
        self.contents = []
        self.file = None
        self.csv_reader = None

        if file_path:
            self.set_file_path(file_path)

    def update(self):
        if not self.file:
            self.contents = []
            return
        self.file.seek(0)
        self.contents = list(self.csv_reader)

    def set_file_path(self, file_path):
        self.file_path = file_path
        if self.file:
            self.file.close()

        self.file = open(file_path, "r", encoding="utf-8", newline="")
```

```
        self.csv_reader = csv.reader(self.file, delimiter=",")
        self.update()

    def get_contents(self):
        self.update()
        return self.contents


class TableWidget(QAbstractTableModel):
    def __init__(self):
        super().__init__()
        self._data = []
        self.watcher = None

    def new_watcher(self, file_path):
        self.watcher = CSVWatcher(file_path)
        self.update()

    def update(self):
        if not self.watcher:
            return
        self.beginResetModel()
        self._data = self.watcher.get_contents()
        self.endResetModel()

    def data(self, index, role):
        if role == Qt.DisplayRole:
            r, c = index.row(), index.column()
            if 0 <= r < len(self._data) and 0 <= c < len(self._data[r]):
                return self._data[r][c]
        return None

    def rowCount(self, parent=None):
        return len(self._data)

    def columnCount(self, parent=None):
        if not self._data:
            return 0
        return max((len(row) for row in self._data), default=0)


# --- Text watcher/view ---

class TextWatcher:
    def __init__(self, file_path=None):
```

```
        self.file_path = file_path
        self.contents = ""

    def update(self):
        if not self.file_path:
            self.contents = ""
            return
        try:
            with open(self.file_path, "r", encoding="utf-8") as f:
                self.contents = f.read()
        except FileNotFoundError:
            self.contents = ""

    def set_file_path(self, file_path):
        self.file_path = file_path
        self.update()

    def get_contents(self):
        self.update()
        return self.contents


class LEDStripVisualizer(QWidget):
    def __init__(self, n_leds=CONST.NUM_LEDS, led_states = None):
        super().__init__()
        self.n_leds = n_leds

        if led_states is None:
            self.rgb = [[0] * 3 for _ in range(n_leds)]
        else:
            self.rgb = led_states

        self.setMinimumHeight(60)

    def set_led(self, index, r, g, b):
        if 0 <= index < self.n_leds:
            self.rgb[index] = [r, g, b]
            self.update()

    def clear_led(self, index):
        if 0 <= index < self.n_leds:
            self.rgb[index] = [0, 0, 0]
            self.update()

    def toggle_led(self, index, r, g, b):
```

```
        if 0 <= index < self.n_leds:
            if self.rgb[index] == [0, 0, 0]:
                self.set_led(index, r, g, b)
            else:
                self.clear_led(index)

    def set_all(self, r, g, b):
        self.rgb = [[r, g, b] for _ in range(self.n_leds)]
        self.update()

    def clear_all(self):
        self.rgb = [[0] * 3 for _ in range(self.n_leds)]
        self.update()

    # QWidget function override, fits the leds nicely in the window, NOTE:
ChatGPT generated
    def paintEvent(self, event):
        p = QPainter(self)
        p.setRenderHint(QPainter.Antialiasing, True)

        w = self.width()
        h = self.height()

        margin = 10
        gap_y = 6  # vertical gap between rows
        usable_w = max(1, w - 2 * margin)
        usable_h = max(1, h - 2 * margin)

        n = self.n_leds
        if n <= 0:
            return

        # Choose a minimum diameter so they remain visible (tune if you want)
        min_d = 6.0

        best = None  # (d, k, rows)
        # Try all possible LEDs-per-row values and choose the one that gives the
biggest diameter
        for k in range(1, n + 1):  # k = leds_per_row
            rows = (n + k - 1) // k

            # diameter limited by height (given rows)
            d_h = (usable_h - gap_y * (rows - 1)) / rows
            if d_h <= 0:
                continue
```

```
            # diameter limited by width (with *some* horizontal gap; we'll
stretch later if needed)
            # For feasibility, require circles alone fit:
            d_w_max = usable_w / k
            d = min(d_h, d_w_max)

            if d >= min_d:
                if best is None or d > best[0]:
                    best = (d, k, rows)

        # If nothing met min_d, fall back to smallest that fits (still no cut-
off)
        if best is None:
            # Pick k that fits height at all, maximize d
            best = (2.0, 1, n)
            for k in range(1, n + 1):
                rows = (n + k - 1) // k
                d_h = (usable_h - gap_y * (rows - 1)) / rows
                if d_h <= 0:
                    continue
                d = min(d_h, usable_w / k)
                if d > best[0]:
                    best = (max(2.0, d), k, rows)

        d, leds_per_row, rows = best

        # Horizontal gap chosen to "reach" the right side without cutting off
circles.
        # If there is space left after placing k circles, distribute it as gap.
        def gap_x_for(k_in_row: int) -> float:
            if k_in_row <= 1:
                return 0.0
            leftover = usable_w - (k_in_row * d)
            return max(0.0, leftover / (k_in_row - 1))

        for i, rgb in enumerate(self.rgb):
            row = i // leds_per_row
            col = i % leds_per_row

            # how many LEDs are in this row? (last row may be partial)
            start = row * leds_per_row
            remaining = n - start
            leds_in_row = min(leds_per_row, remaining)
```

```
            gx = gap_x_for(leds_in_row)

            # Row width now fills usable_w (or is centered if leds_in_row==1)
            row_width = leds_in_row * d + (leds_in_row - 1) * gx
            x0 = margin + (usable_w - row_width) / 2.0

            x = x0 + col * (d + gx)
            y = margin + row * (d + gap_y)

            p.setBrush(QColor(int(rgb[0]), int(rgb[1]), int(rgb[2])))
            p.setPen(Qt.black)
            p.drawEllipse(QRectF(x, y, d, d))

            p.setPen(Qt.white if sum(rgb) < 200 else Qt.black)
            p.drawText(QRectF(x, y, d, d), Qt.AlignCenter, str(i))




class StartupDialog(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Select Mode")

        central = QWidget()
        layout = QHBoxLayout(central)

        self.btns = []
        for mode in CONST.MODES:
            temp = QPushButton(mode)
            temp.clicked.connect(lambda _, m=mode: self.setmode(m))
            self.btns.append(temp)
            layout.addWidget(temp)


        self.setCentralWidget(central)
        self.resize(200, 50)

    def show_window(self):
        self.loop = QEventLoop()
        self.show()
```

```
        print("Waiting for mode selection")
        self.loop.exec()

    def setmode(self, mode):
        global MODE
        if mode in CONST.MODES:
            self.selected_mode = mode
            MODE = mode
        else:
            raise ValueError("INVALID MODE SELECTED UPON STARTUP")

        self.loop.quit()

        self.close()


# --- Main window with side-by-side widgets ---

class MainWindow(QMainWindow):
    def __init__(self, log_path: str, dlog_path: str, mode: str):
        super().__init__()
        self.setWindowTitle(f"Soft Front Panel - MODE: {mode}")

        # File/text viewer labels
        self.log_label = QLabel(f"Log File \n\nPath: {log_path}")
        self.dlog_label = QLabel(f"Detailed Log File \n\nPath: {dlog_path}")
        command_label = QLabel(f"SFP User Commands & Error Reporting")

        label_layout = QHBoxLayout()
        label_layout.addWidget(self.log_label)
        label_layout.addWidget(self.dlog_label)
        label_layout.addWidget(command_label)

        # log
        self.csv_model = TableWidget()
        self.csv_model.new_watcher(log_path)
        self.csv_view = QTableView()
        self.csv_view.setModel(self.csv_model)

        # Detailed log
        self.text_watcher = TextWatcher(dlog_path)
        self.dlog = QPlainTextEdit()
        self.dlog.setReadOnly(True)
        self.dlog.setPlainText(self.text_watcher.get_contents())
```

```
        # User Command log
        self.command_log = QPlainTextEdit()
        self.command_log.setReadOnly(True)
        self.command_log.setPlainText(f"Mode Selected: {mode}")


        # Outgoing led driver commands

        # Expected LED State

        central = QWidget()
        root_layout = QVBoxLayout(central)

        # Layout for text viewers
        viewer_layout = QHBoxLayout()
        viewer_layout.addWidget(self.csv_view, 1)
        viewer_layout.addWidget(self.dlog, 1)
        viewer_layout.addWidget(self.command_log, 1)


        # Timers, provides updates for text viewers
        self.csv_timer = QTimer(self)
        self.csv_timer.timeout.connect(self.csv_model.update)
        self.csv_timer.start(500)

        self.text_timer = QTimer(self)
        self.text_timer.timeout.connect(self.update_text_view)
        self.text_timer.start(500)

        # Led visualizer
        self.strip = LEDStripVisualizer()


        # buttons
        self.new_trial_button = QPushButton("New Trial")
        self.start_button = QPushButton("Start Trial")
        self.style_combobox = QComboBox()
        self.style_combobox.addItems(["FillTrail", "RainbowTrail",
"SurroundingLEDsLit", "Rainbow", "Ember", "SpeedHeatmap"])

        self.set_style_button = QPushButton("Set Style")

        self.all_LED_on_button = QPushButton("All LEDs On")
        self.all_LED_on_button.clicked.connect(lambda: self.strip.set_all(255, 0,
0))
```

```
        self.all_LED_off_button = QPushButton("All LEDs Off")
        self.all_LED_off_button.clicked.connect(self.strip.clear_all)

        self.manual_LED_spinbox = QSpinBox()
        self.manual_LED_spinbox.setRange(0, CONST.NUM_LEDS -1)

        self.manual_LED_button = QPushButton("Toggle Specified LED")
        self.manual_LED_button.clicked.connect(lambda:
self.strip.toggle_led(self.manual_LED_spinbox.value(), 255, 0, 0))

        btns = [self.new_trial_button, self.start_button, self.style_combobox,
self.set_style_button, self.all_LED_on_button, self.all_LED_off_button,
self.manual_LED_button, self.manual_LED_spinbox]

        button_row = QHBoxLayout()
        for btn in btns:
            button_row.addWidget(btn)

        root_layout.addLayout(label_layout)
        root_layout.addLayout(viewer_layout)
        root_layout.addWidget(self.strip)
        root_layout.addLayout(button_row)

        self.setCentralWidget(central)

    def set_log_paths(self, log_path: str, dlog_path: str):

        self.log_label.setText(f"Log File\n\nPath: {log_path}")
        self.dlog_label.setText(f"Detailed Log File\n\nPath: {dlog_path}")

        # Update existing watcher/model
        self.csv_model.new_watcher(log_path)

        # Update existing text watcher
        self.text_watcher.set_file_path(dlog_path)
        self.dlog.setPlainText(self.text_watcher.get_contents())

        self.update_command_log("GUI watchers repointed to new log files.")


    def update_text_view(self):
        new_text = self.text_watcher.get_contents()
        # Minimal optimization: only repaint if changed
        if new_text != self.dlog.toPlainText():
```

```
            self.dlog.setPlainText(new_text)

    def update_command_log(self, to_add):
        self.command_log.appendPlainText(to_add)


if __name__ == "__main__":
    app = QApplication(sys.argv)

    win = StartupDialog()
    win.show_window()
    print(MODE)


    win2 = MainWindow(
        log_path="logs/Log_11-14-25_16-03-26.csv",
        dlog_path="logs/test.txt",
    )

    win2.resize(1200, 650)
    win2.showMaximized()

    sys.exit(app.exec())
```

ModuleIntegration.py

```
# ModuleIntegration.py
# Manages connections to GUI & starts threads
# Author: Noah Sergent

from __future__ import annotations
from SerialHandler import SerialHandler as sh
from Logger import Logger
from ConfigManager import ConfigManager
from MessageProcessor import MessageProcessor
from GUI import MainWindow
import Error
from PySide6.QtCore import QObject
from queue import Queue
from Animator import Animator
import AnimationStyles

from dataclasses import dataclass
import threading
from typing import Optional, List, Tuple
```

```
from PySide6.QtCore import QObject, Signal, Slot

NUM_LEDS = 144

RGB = Tuple[int, int, int]

@dataclass
class CoordinatorState():
    trial_running: bool = False

    led_rgb: Optional[List[RGB]] = None



class Coordinator(QObject):

    E_SOURCE = "Coordinator"

    # Signals for GUI
    command_text = Signal(str)
    status_text = Signal(str)
    log_paths_changed = Signal(str,str)

    led_set = Signal(int, int, int, int)
    led_toggle = Signal(int, int, int, int)
    led_all = Signal(int, int, int)
    led_clear = Signal()

    ascii_message = Signal(str)
    daq_row_logged = Signal(list)


    def __init__(self, global_message_queue: Queue, message_processor:
MessageProcessor | None = None, daq_sh: sh | None = None, led_sh: sh | None =
None, logger: Logger | None = None, conf: ConfigManager | None = None, animator:
Animator | None = None):
        super().__init__()
        self.daq_sh = daq_sh
        self.led_sh = led_sh
        self.logger = logger
        self.confman = conf
        self.shared_queue = global_message_queue
        self.message_processor = message_processor
        self.animator = animator
```

```
        self.state = CoordinatorState(trial_running=False, led_rgb=[[0, 0, 0] for
_ in range(NUM_LEDS)]
)
        self._stop_event = threading.Event()
        self._daq_thread: Optional[threading.Thread] = None
        self._led_thread: Optional[threading.Thread] = None

    # Serial Integration

    def attach_sh(self, *, daq_sh: sh | None = None, led_sh: sh | None = None):
        self.daq_sh = daq_sh
        self.led_sh = led_sh

        if self.daq_sh is not None:
            self.daq_sh.set_logger(self.logger)

        if self.led_sh is not None:
            self.led_sh.set_logger(self.logger)


        self.command_text.emit("Serial Devices Attached")

    def emit_command_text(self, text: str):
        self.command_text.emit(text)

    def start_serial_threads(self):

        if self.daq_sh:
            self.daq_sh.start()
            self.command_text.emit("DAQ SerialHandler threads started.")

        if self.led_sh:
            self.led_sh.start()
            self.command_text.emit("LED controller SerialHandler threads
started.")

        if self.message_processor:
            self.message_processor.set_incoming_msg_queue(self.shared_queue)
            self.message_processor.start()
            self.command_text.emit("MessageProcessor threads started.")


    # Logging

    @Slot()
```

```
    def new_trial(self):
        """Create new log files by setting config flag and creating new logger
        """

        self.command_text.emit("Starting new trial...")

        self.confman.new_trial_set()

        self.logger = Logger(self.confman)

        if self.daq_sh:
            self.daq_sh.set_logger(self.logger)
            self.daq_logger = self.logger

        if self.led_sh:
            self.led_sh.set_logger(self.logger)
            self.led_logger = self.logger

        self.log_paths_changed.emit(self.logger.get_log_path(),
self.logger.get_dlog_path())
        self.command_text.emit("New trial logs created")

        # Set new trial value to false
        self.confman.new_trial_set(False)
        self.confman.save_configuration()

    @Slot()
    def start_trial(self):
        self.state.trial_running = True
        self.command_text.emit("Trial Started TODO IMPLEMENT") # TODO IMPLEMENT
IN DAQ

    @Slot()
    def stop_trial(self):
        self.state.trial_running = False
        self.command_text.emit("Trial Stopped TODO IMPLEMENT") # TODO IMPLEMENT
IN DAQ

    @Slot()
    def reset_daq(self):
        self.command_text.emit("DAQ Reset Requested. TODO IMPLEMENT")# TODO
IMPLEMENT IN DAQ

    STYLE_MAP = {
        "FillTrail": AnimationStyles.FillTrail,
```

```
        "RainbowTrail": AnimationStyles.RainbowTrail,
        "SurroundingLEDsLit": AnimationStyles.SurroundingLEDsLit,
        "Rainbow": AnimationStyles.Rainbow,
        "Ember": AnimationStyles.Ember,
        "SpeedHeatmap": AnimationStyles.SpeedHeatmap,
    }

    @Slot(str)
    def set_animation_style(self, style_name: str):
        style_cls = self.STYLE_MAP.get(style_name)
        if style_cls is None:
            self.command_text.emit(f"Unknown style: {style_name}")
            return
        if self.animator:
            self.animator.set_style(style_cls())
            self.command_text.emit(f"Animation style set to: {style_name}")
        else:
            self.command_text.emit("No animator available!")
            Error.report("No animator available!", self.E_SOURCE)

    # LED commands

    @Slot()
    def all_led_on(self):
        self.command_text.emit("Set All LEDs to ON")
        if self.led_sh:
            self.animator.all_on()
        else:
            self.command_text.emit("No LED SerialHandler attached!")
            Error.report("No LED SerialHandler attached!", self.E_SOURCE)
        self.led_all.emit(255, 0, 0)

    @Slot()
    def all_led_off(self):
        self.command_text.emit("Set All LEDs to OFF")
        if self.led_sh:
            self.animator.all_off()
        else:
            self.command_text.emit("No LED SerialHandler attached!")
            Error.report("No LED SerialHandler attached!", self.E_SOURCE)
        self.led_clear.emit()

    @Slot(int)
    def set_single_led(self, index: int, r: int = 255, g: int =0, b: int = 0):
        self.command_text.emit(f"LED set: {index} ({r}, {g}, {b})")
```

```
        if self.led_sh:
            self.message_processor.toggle_led(index)
        else:
            self.command_text.emit("No LED SerialHandler attached!")
            Error.report("No LED SerialHandler attached!", self.E_SOURCE)
        self.led_set.emit(index, r, g, b)




def connect_coordinator_to_gui(coord: Coordinator, gui: MainWindow):
    coord.command_text.connect(gui.update_command_log)
    coord.led_set.connect(lambda i, r, g, b: gui.strip.set_led(i, r, g, b))
    coord.led_all.connect(lambda r, g, b: gui.strip.set_all(r, g, b))
    coord.led_clear.connect(gui.strip.clear_all)
    coord.log_paths_changed.connect(gui.set_log_paths)

    gui.new_trial_button.clicked.connect(coord.new_trial)
    gui.start_button.clicked.connect(coord.start_trial)
    gui.set_style_button.clicked.connect(lambda:
coord.set_animation_style(gui.style_combobox.currentText()))

    gui.all_LED_off_button.clicked.connect(coord.all_led_off)
    gui.all_LED_on_button.clicked.connect(coord.all_led_on)
    gui.manual_LED_button.clicked.connect(lambda:
coord.set_single_led(gui.manual_LED_spinbox.value()))
```

Logger.py

```
# Logger.py
# Contains functions for logging, creates and manages paths to log files
# Author: Noah Sergent

import typing
import time
import os
import csv

import GenFunct
from ConfigManager import ConfigManager
import CONST



# def print_no_newline(to_print):
#     print(to_print, )
```

```
class Logger():

    def __init__(self, configmanager: ConfigManager = None,
csv_header=CONST.DEFAULT_CSV_HEADER):

        # If an instance of ConfigManager is not passed to constructor, create a
new instance.
        if configmanager == None:
            self.configmanager = ConfigManager()
        else:
            self.configmanager = configmanager


        self.logdir = os.path.join(self.configmanager.config_dir_abspath,
*self.configmanager.config_dict["Relative Log Directory Path"])

        need_new_logs = self.check_if_need_new_log()

        # Create new log files if this is a new trial, the log files aren't
specified in config file, or the script cannot find the specified logfiles
        if need_new_logs:
            self.create_log_files(self.logdir, csv_header)
        else:
            logfile_path = os.path.join(self.logdir,
self.configmanager.config_dict["Log File Name"])
            detailed_logfile_path = os.path.join(self.logdir,
self.configmanager.config_dict["Detailed Log Name"])

            self.logfile = open(logfile_path, 'a', encoding="utf-8", newline='')
            self.logfile_writer = csv.writer(self.logfile)
            self.detailed_logfile = open(detailed_logfile_path, 'a',
encoding="utf-8", newline='')

    def create_log_files(self, dir_path, csv_header):
        timestamp = GenFunct.get_timestamp(µs_scale=False,
filename_friendly=True)
        logfile_name = f"Log_{timestamp}.csv"
        detailed_logfile_name = f"Detailed_Log_{timestamp}.txt"

        self.logfile_path = os.path.join(dir_path, logfile_name)
        self.detailed_logfile_path = os.path.join(dir_path,
detailed_logfile_name)

        # Create files
```

```
        self.logfile = open(self.logfile_path, 'w', encoding="utf-8", newline='')
        self.detailed_logfile = open(self.detailed_logfile_path, 'w',
encoding="utf-8", newline='')

        # Prime Files
        self.logfile_writer = csv.writer(self.logfile)
        self.logfile_writer.writerow(csv_header)
        self.detailed_logfile.write(f"Created: {timestamp}")
        self.detailed_logfile.write("\nConfiguration:\n" +
str(self.configmanager.config_dict) + '\n')

        self.logfile.flush()
        self.detailed_logfile.flush()

        # Update Config
        self.configmanager.config_dict["Log File Name"] = logfile_name
        self.configmanager.config_dict["Detailed Log Name"] =
detailed_logfile_name

        # Sav Updates
        self.configmanager.save_configuration()

    def check_if_need_new_log(self) -> bool:

        # if the log directory doesn't exist, create one.
        if not os.path.isdir(self.logdir):
            os.mkdir(self.logdir)

        need_new_log = (
            self.configmanager.config_dict["New Trial"]
            or not GenFunct.check_exists(self.logdir,
self.configmanager.config_dict["Log File Name"])
            or not GenFunct.check_exists(self.logdir,
self.configmanager.config_dict["Detailed Log Name"])
        )
        return need_new_log


    def log(self, row_to_write):
        self.logfile_writer.writerow(row_to_write)
        self.logfile.flush()
        self.dlog(f"Row added to log:\n{row_to_write}", start_char='\n',
end_char='\n')

    def dlog(self, to_write, add_timestamp=True, start_char='', end_char='\n'):
```

```
        timestamp = f"{GenFunct.get_timestamp(µs_scale=True)}: " if add_timestamp
else ''
        to_write = f"{start_char}{timestamp}{to_write}{end_char}"
        self.detailed_logfile.write(to_write)
        self.detailed_logfile.flush()
        print(start_char + to_write, end=end_char)

    def save_backup(self): #TODO implement
        """Save a backup of the current logs folder, files already present will
be duplicated with an appended number.
           NOTE: This backup directory should ONLY be appended to, no deletions
or edits!
        """
        pass

    def get_log_path(self):
        return self.logfile_path

    def get_dlog_path(self):
        return self.detailed_logfile_path

if __name__ == "__main__":
    logger = Logger()
```

GenFunct.py

```
# GenFunct.py
# Contains functions that are reused across multiple modules, or do not fit
within the scope of any module
# Author: Noah Sergent

import os
from datetime import datetime



def get_timestamp(µs_scale=False, filename_friendly=False):
    """Gets the current time in the following format YY-DD-MM_HH:mm:SS:µµµµµµ

    Args:
        µs_scale (bool, optional): Determines if microseconds will be included in
the timestamp. Defaults to False
        filename_friendly (bool, optional): Will replace ':' with '-' in
timestamp if true for sake of windows filenames. Defaults to False.
```

```
    Returns:
        str: Current date & time.
    """
    timestamp_format = r"%m-%d-%y %H:%M:%S"
    if µs_scale: timestamp_format += r":%f"
    if filename_friendly: timestamp_format = timestamp_format.replace(':', '-
').replace(' ', '_')
    return datetime.now().strftime(timestamp_format)

def check_exists(*path) -> bool:
    """Checks if a file or directory exists, can accept any number of arguments
    """
    try:
        return os.path.exists(os.path.join(*path))
    except TypeError:
        return False

def to_seconds(x):
    return (int.from_bytes(x, byteorder="big"))/2e6
```

Error.py

```
#Error.py
# Centralized error reporting for project, reports to GUI and detailed log
# Author: Noah Sergent

logger = None
coord = None

def set_recipients(new_logger, new_coordinator=None):
    global logger, coord
    logger = new_logger
    coord = new_coordinator

def warn(message, source: str | None = None):
    global logger, coord
    print(f"{warn_pretext(source)} {message}")

    if logger:
        logger.dlog(f"{warn_pretext(source)} {message}")

    if coord:
        coord.emit_command_text(f"{warn_pretext(source)} {message}")

def report(message, source: str | None = None):
    global logger, coord
```

```
    print(f"{report_pretext(source)} {message}")

    if logger:
        logger.dlog(f"{report_pretext(source)} {message}")

    if coord:
        coord.emit_command_text(f"{report_pretext(source)} {message}")

def event(message, source: str | None = None):
    global logger, coord
    print(f"{event_pretext(source)} {message}")

    if logger:
        logger.dlog(f"{event_pretext(source)} {message}")

    if coord:
        coord.emit_command_text(f"{event_pretext(source)} {message}")


def warn_pretext(source=None):
    return f"WARN: {f"({source})" if source else ""}"

def report_pretext(source=None):
    return f"ERROR: {f"({source})" if source else ""}"

def event_pretext(source=None):
    return f"EVENT: {f"({source})" if source else ""}"
```

CONST.py

```
"""CONST.py

Holds global immutable constants for the project
NOTE: Maybe move this to config.json???????
# Author: Noah Sergent
"""

# Placeholder
ANY = None

# GLOBAL CONSTANTS
DAQ = "DAQ"
LED_CONTROLLER = "LED Controller"

    # UART Baud Rates
```

```
DAQ_BAUD = 115200
LED_DRIVER_BAUD = 115200

    # UART COM Ports
DAQ_COM = "COM8"
LED_DRIVER_COM = "COM5"

    # UART Protocol Constants
STX = 0x02
ETX = 0x03
EOT = 0x04
ENQ = 0x05
ACK = 0x06

    # Message Types
CMD = 'C'
ASCII = 'M'
STATUS = 'S'
DATA = 'D'

    # Message Descriptors
START_TIME_DESCRIPTOR = 'T'
DURATION_DESCRIPTOR = 'D'
SINGLE_LED_DESCRIPTOR = 'L'
SET_LED_DESCRIPTOR = 'S'
CLEAR_LED_DESCRIPTOR = 'C'
ALL_LED_OFF_DESCRIPTOR = 'X'
ALL_LED_ON_DESCRIPTOR =  'O'
TOGGLE_LED_DESCRIPTOR = 'T'
LED_DATA_DESCRIPTOR = 'D'

    # Modes
DAQ_ONLY = "DAQ Only"
LED_ONLY = "Led Controller Only"
FULL = "Full System"
SFP_DEBUG = "SFP Debug"
LATENCY_TEST = "Latency Test"
MODES = [DAQ_ONLY, LED_ONLY, FULL, SFP_DEBUG, LATENCY_TEST]

    # DAQ Constants
MINIMUM_DURATION = 0.001
MINIMUM_TIME_BETWEEN_DATA = 0
DEBOUNCE_S = 0.01
F_CPU_UNO = 16e6
N_PRESCALER_UNO = 8
```

```
TCNT_SCALAR_UNO = (N_PRESCALER_UNO / F_CPU_UNO)

    #MessageProcessor constants
RUN_TIMEOUT_S = 10.0
RUN_COOLDOWN_S = 5.0

    # LEDs
NUM_LEDS = 144

LED_INTENSITY_HIGH = 255
LED_INTENSITY_MEDIUM = 75
LED_INTENSITY_LOW = 25
LED_INTENSITY_MULTIPLIER = 0.25


    # Predictions
SCHEDULE_OFFSET_S = -0.01
INCLUDE_PREDICTION_IN_LOG = False

SENSOR_STRETCHES = {
    0: "sensor_0_to_1",
    1: "sensor_1_to_2",
    2: "sensor_2_to_3",
    3: "sensor_3_to_4",
}

    # Distances

LED_DISTANCES = {
    0: 0
}
#INITAL TESTING LED DISTANCES
LED_DISTANCES[0] = 0
for i in range(1, NUM_LEDS):
    LED_DISTANCES[i] = LED_DISTANCES[i-1] + 7

SENSOR_DISTANCES = {
    0: 0,
    1: 145.25,
    2: 378.11,
    3: 606.813,
    4: 747.063
}

    # Log Header
```

```
if INCLUDE_PREDICTION_IN_LOG:
    DEFAULT_CSV_HEADER = ["run_id", "event_id", "timestamp", "sensor_id",
"break_duration", "prediction"]
else:
    DEFAULT_CSV_HEADER = ["run_id", "event_id", "timestamp", "sensor_id",
"break_duration"]

    # Animation
REFRESH = 120
DEFAULT_COLOR = 1 # 1 red, 2 blue, 3, green

# from pprint import pprint

# if __name__ == "__main__":
#     pprint(LED_DISTANCES)
```

ConfigManager.py

```
# ConfigManager.py
# Author: Noah Sergent

import json
import os

class ConfigManager():

    def __init__(self, new_trial=False, config_file = "config.json"):
        self.config_file = config_file

        with open(self.config_file, 'r') as jsonfile:
            self.config_dict = json.load(jsonfile)

        self.config_file_abspath = os.path.abspath(config_file)
        self.config_dir_abspath = os.path.dirname(self.config_file_abspath)

    def save_configuration(self):
        with open(self.config_file, 'w') as jsonfile:
            json.dump(self.config_dict, jsonfile, indent=4)

    def new_trial_set(self, val: bool | None = True):
        self.config_dict["New Trial"] = val

    def get_sensor_position(self, ID):
        sensor_dict = self.config_dict["Sensors"].get(str(ID), None)
        if sensor_dict:
```

```
            return sensor_dict["Position"]

        return None

    def get_last_sensor_id(self) -> int | None:
        for sensor_id, sensor_dict in self.config_dict["Sensors"].items():
            if sensor_dict.get("Position") == "Last":
                return int(sensor_id)
        return None
```

Animator.py

```
# Animator.py
# Author: Noah Sergent

from SerialHandler import SerialHandler
import CONST
import time
import threading
import Error
import queue
import Logger
from AnimationStyles import AnimationStyle
import AnimationStyles

class Animator():
    def __init__(self, refresh_rate_hz: int, led_sh: SerialHandler):
        self.sh = led_sh
        self.refresh_rate_seconds = 1 / refresh_rate_hz
        self.RGB = [[0, 0, 0] for _ in range(CONST.NUM_LEDS + 10)]
        self._worker_thread = threading.Thread(target=self._update_worker,
daemon=True, name="AnimatorWorker")
        self.surrounding_leds_lit = True
        self._style = AnimationStyles.SurroundingLEDsLit()
        self.actve_index = 0

    def set_style(self, style: AnimationStyle):
        self._style = style


    def _update_worker(self):
        E_SOURCE = "Animator Worker"
        while True:
            time.sleep(self.refresh_rate_seconds)
            temp = []
```

```
            for i in range(CONST.NUM_LEDS):
                temp.extend(self.RGB[i])

            if self._style:
                self._style.tick(self.RGB, self.actve_index)

            try:
                self.sh.send_frame(
                    ord('C'),
                    0xFF,
                    ord(CONST.LED_DATA_DESCRIPTOR),
                    temp,
                    ignore_length=True
                )
            except Exception as e:
                Error.report(f"{e}", E_SOURCE)

    def start(self):
        self._worker_thread.start()

    def set_led(self, index, r=int(CONST.LED_INTENSITY_HIGH *
CONST.LED_INTENSITY_MULTIPLIER), g=0, b=0):

        if hasattr(self._style, "set_led"):
            self._style.set_led(self.RGB, index)
        else:
            self.actve_index = index
            self.RGB = [[0, 0, 0] for _ in range(CONST.NUM_LEDS+10)]
            self.RGB[index] = [0, 0, int(CONST.LED_INTENSITY_HIGH *
CONST.LED_INTENSITY_MULTIPLIER)]

        if self._style:
            self._style.apply(self.RGB, index)

    def toggle_led(self, index, r=CONST.LED_INTENSITY_HIGH, g=0, b=0):
        if self.RGB[index] != [0, 0, 0]:
            self.RGB[index] = [r, g, b]
        else:
            self.RGB[index] = [0, 0, 0]

    def unit_test(self):
        while(1):
            self.RGB[1] = [0, 0, 0]
            self.RGB[0] = [100, 0, 0]
            time.sleep(0.5)
```

```
            self.RGB[0] = [0, 0, 0]
            self.RGB[1] = [100, 0, 0]
            time.sleep(0.5)

    def all_on(self):
        Error.event("rgb set on","ALLON")
        self.RGB = [[0, 0, int(255 * CONST.LED_INTENSITY_MULTIPLIER)] for _ in
range(CONST.NUM_LEDS)]

    def all_off(self):
        self.RGB = [[0, 0, 0] for _ in range(CONST.NUM_LEDS)]
        if hasattr(self._style, 'on_completion'):
            self._style.on_completion()
```

AnimationStyles.py

```
# AnimationStyles.py
# Author: Noah Sergent

import colorsys
from abc import ABC, abstractmethod
import CONST
import Error
import time

E_SOURCE = "AnimationStyles"

def rainbow_colors(n=256):
    """Generate n RGB values cycling through the rainbow."""
    return [
        [int(c * 255 * CONST.LED_INTENSITY_MULTIPLIER) for c in
colorsys.hsv_to_rgb(i / n, 1.0, 1.0)]
        for i in range(n)
    ]

if __name__ == "__main__":
    print(rainbow_colors(10))

class AnimationStyle(ABC):

    @abstractmethod
    def apply(self, RGB: list[list[int]], index: int):
        pass

    def tick(self, RGB: list[list[int]], index: int):
```

```
        pass

class SurroundingLEDsLit(AnimationStyle):

    def __init__(self, num_leds: int = CONST.NUM_LEDS):
        self.surrounding_leds_high = 1
        self.surrounding_leds_med = 1
        self.surrounding_leds_low = 1

    def apply(self, RGB, index):
        RGB[index+self.surrounding_leds_high] = [0, 0,
int(CONST.LED_INTENSITY_HIGH * CONST.LED_INTENSITY_MULTIPLIER)]
        RGB[index-self.surrounding_leds_high] = [0, 0,
int(CONST.LED_INTENSITY_HIGH * CONST.LED_INTENSITY_MULTIPLIER)]

        RGB[index+self.surrounding_leds_high+self.surrounding_leds_med] = [0, 0,
int(CONST.LED_INTENSITY_MEDIUM * CONST.LED_INTENSITY_MULTIPLIER)]
        RGB[index-self.surrounding_leds_high-self.surrounding_leds_med] = [0, 0,
int(CONST.LED_INTENSITY_MEDIUM * CONST.LED_INTENSITY_MULTIPLIER)]

        RGB[index+self.surrounding_leds_high+self.surrounding_leds_med+self.surro
unding_leds_low] = [0,0, int(CONST.LED_INTENSITY_LOW *
CONST.LED_INTENSITY_MULTIPLIER)]
        RGB[index-self.surrounding_leds_high-self.surrounding_leds_med-
self.surrounding_leds_low] = [0,0, int(CONST.LED_INTENSITY_LOW *
CONST.LED_INTENSITY_MULTIPLIER)]

    def tick(self, RGB, index):
        pass

class FillTrail(AnimationStyle):

    def __init__(self, num_leds: int = CONST.NUM_LEDS):
        self.fill_color = [int(255 * CONST.LED_INTENSITY_MULTIPLIER), 0, 0]
        self.last_RGB = None

    def set_led(self, RGB, index):
        RGB[index] = self.fill_color

    def apply(self, RGB, index):
        pass

    def tick(self, RGB, index):
        pass
```

```
class RainbowTrail(AnimationStyle):
    def __init__(self, num_leds: int = CONST.NUM_LEDS):
        self.colors = rainbow_colors(10)*100

    def set_led(self, RGB, index):
        RGB[index] = self.colors[index]

    def apply(self, RGB, index):
        pass

    def tick(self, RGB, index):
        pass

class Ember(AnimationStyle): #TODO: this aint workin
    def __init__(self, num_leds: int = CONST.NUM_LEDS):
        self.ember_color = [255, 255, 255]
        self.ember_interval = 10
        self.flicker_interval = 100
        self.ember_indexes = set()
        self.ticks = 0

    def apply(self, RGB, index):
        if index % 10 == 0:
            self.ember_indexes.add(index)

    def tick(self, RGB, _):
        self.ticks += 1
        if self.ticks % 10 == 0:
            for index in self.ember_indexes:
                if RGB[index] != [0, 0, 0]:
                    RGB[index] = [0, 0, 0]       # on → off
                else:
                    RGB[index] = [255, 255, 0]   # off → on

    def on_completion(self):
        Error.event(str(self.ember_indexes) + "\n" + str(self.ticks), E_SOURCE)

class Rainbow(AnimationStyle):
    def __init__(self, num_leds: int = CONST.NUM_LEDS):
        self.colors = []
        for i in range(144):
            h = i / 50
            r, g, b = colorsys.hsv_to_rgb(h, 1.0, CONST.LED_INTENSITY_HIGH / 255)
            self.colors.append([int(r * 255), int(g * 255), int(b * 255)])
```

```
    def apply(self, RGB, index):
        RGB[index] = self.colors[index]

class SpeedHeatmap(AnimationStyle):
    """Colors each LED based on the ball's speed at that point.
    Blue = slow, red = fast. Recolors the entire trail whenever
    the observed speed range expands so the heatmap stays normalized.
    """

    def __init__(self, num_leds: int = CONST.NUM_LEDS):
        self._last_index = None
        self._last_time = None
        self._min_speed = float('inf')
        self._max_speed = 0.0
        self._speeds: dict[int, float] = {}

    def set_led(self, RGB, index):
        now = time.perf_counter()

        if self._last_index is not None and self._last_time is not None:
            dt = now - self._last_time
            dist = abs(CONST.LED_DISTANCES[index] -
CONST.LED_DISTANCES[self._last_index])

            if dt > 0 and dist > 0:
                speed = dist / dt
                self._speeds[index] = speed

                range_changed = False
                if speed < self._min_speed:
                    self._min_speed = speed
                    range_changed = True
                if speed > self._max_speed:
                    self._max_speed = speed
                    range_changed = True

                if range_changed:
                    self._recolor_all(RGB)
                else:
                    RGB[index] = self._speed_to_color(speed)
            else:
                RGB[index] = self._speed_to_color(self._min_speed)
        else:
            # First LED in the run, no speed yet — neutral white dot
            RGB[index] = [int(255 * CONST.LED_INTENSITY_MULTIPLIER)] * 3
```

```
        self._last_index = index
        self._last_time = now

    def _speed_to_color(self, speed):
        if self._max_speed == self._min_speed:
            norm = 0.5
        else:
            norm = (speed - self._min_speed) / (self._max_speed -
self._min_speed)

        # blue (hue 0.66) → red (hue 0.0)
        hue = 0.66 * (1.0 - norm)
        r, g, b = colorsys.hsv_to_rgb(hue, 1.0, CONST.LED_INTENSITY_MULTIPLIER)
        return [int(r * 255), int(g * 255), int(b * 255)]

    def _recolor_all(self, RGB):
        for idx, speed in self._speeds.items():
            RGB[idx] = self._speed_to_color(speed)

    def apply(self, RGB, index):
        pass

    def tick(self, RGB, index):
        pass
```

LED Controller (C++)

Arduino IDE Sketch

```
/* Quick and dirty led driver using arduino ide, hopefully switch to baremetal
version at some point*/
// Author: Noah Sergent


class WrappingIndex {
    public:
        uint16_t value;
        uint16_t max;

        WrappingIndex() : value(0){};
        WrappingIndex(uint16_t init, uint16_t max) : value(init), max(max){};

        inline volatile WrappingIndex& operator++() volatile {
            value = value + 1;
            if (value > max){
```

```
            value = 0;
        }
        return *this;
    }

    inline WrappingIndex operator++(int) volatile {
        WrappingIndex temp(value, max);
        ++(*this);
        return temp;
    }

    inline volatile WrappingIndex& operator=(uint16_t newval) volatile {
        value = newval;
        return *this;
    }

    inline operator uint16_t() const volatile {return value;}


};
#include <FastLED.h>

#define NUM_LEDS 144
#define EXPECTED_PAYLOAD_LEN 432
CRGB leds[NUM_LEDS];

uint32_t active_led_index = 0;

#define TX_BUFSIZE 256
#define RX_BUFSIZE 512

uint8_t rx_buf[RX_BUFSIZE] = {0};
uint8_t tx_buf[TX_BUFSIZE] = {0};

uint16_t tx_head = 0;
uint16_t tx_tail = 0;
WrappingIndex rx_head = WrappingIndex(0, 511);
WrappingIndex rx_tail = WrappingIndex(0, 511);

bool tx_buf_cleared = true;
bool incoming_message_complete = false;

//ItsyBitsy ID
uint8_t ID = 0x02;
```

```
// Standard ASCII values
const uint8_t ENQ = 0x05;
const uint8_t ACK = 0x06;
const uint8_t STX = 0x02;
const uint8_t ETX = 0x03;
const uint8_t EOT = 0x04;

// Message type values
const uint8_t MESSAGE = 'M';
const uint8_t DATA = 'D';
const uint8_t COMMAND = 'C';

// Descriptor values
const uint8_t SINGLE_LED_DESCRIPTOR = 'L';
const uint8_t SET_LED_DESCRIPTOR = 'S';
const uint8_t CLEAR_LED_DESCRIPTOR = 'C';
const uint8_t ALL_LED_OFF_DESCRIPTOR = 'X';
const uint8_t ALL_LED_ON_DESCRIPTOR =  'O';
const uint8_t TOGGLE_LED_DESCRIPTOR = 'T';
const uint8_t LED_DATA_DESCRIPTOR = 'D';

struct MsgStruct {
  uint8_t msg_type;
  uint8_t id;
  uint8_t descriptor;
  uint8_t payload_length;
  uint8_t payload_index;   // RX: index into rx_buf where payload begins
  uint8_t *payload;        // TX: pointer to payload bytes
};



MsgStruct incoming;
MsgStruct outgoing;

uint8_t tx_payload_buf[128] = {0};

void readSerial();
void unpackMessage(MsgStruct *message);
void processMessage(MsgStruct *message);
void enqueueMsg(MsgStruct *message, uint8_t message_length);
void flushTX();
```

```
static inline uint8_t tx_next(uint8_t x) { return (uint8_t)(x + 1); } // 256-size
wrap via uint8_t overflow



void setup() {
  Serial.begin(115200);
  while (!Serial) {}

  FastLED.addLeds<WS2812B, 9>(leds, NUM_LEDS);
  FastLED.clear(true);

  // prime TX buffer
  tx_buf[tx_head++] = ENQ;
  tx_buf_cleared = true;

  // Make sure all LEDs are off
  for (uint32_t i = 0; i < NUM_LEDS; i++) {
    leds[i] = CRGB::Black;
  }
  FastLED.show();

  sendASCIIMessage("Ready!", 0x00);
}

// the loop function runs over and over again forever
void loop() {
  readSerial();

  if (incoming_message_complete) {
    rx_tail = 0;

    if (rx_buf[rx_tail] == ENQ) rx_tail++;
    if (rx_buf[rx_tail] == STX) rx_tail++;

    unpackMessage(&incoming);
    processMessage(&incoming);

    // Reset for next message
    incoming_message_complete = false;
    rx_head = 0;
    rx_tail = 0;
  }
}
```

```
void sendASCIIMessage(const char* msg, uint8_t value) {

  // Copy base string into payload buffer
  uint8_t len = 0;
  while (msg[len] != '\0' && len < (sizeof(tx_payload_buf) - 4)) {
    tx_payload_buf[len] = (uint8_t)msg[len];
    len++;
  }

  // Convert uint8_t to ASCII and append
  uint8_t v = value;

  if (v >= 100) {
    tx_payload_buf[len++] = '0' + (v / 100);
    v %= 100;
    tx_payload_buf[len++] = '0' + (v / 10);
    tx_payload_buf[len++] = '0' + (v % 10);
  }
  else if (v >= 10) {
    tx_payload_buf[len++] = '0' + (v / 10);
    tx_payload_buf[len++] = '0' + (v % 10);
  }
  else {
    tx_payload_buf[len++] = '0' + v;
  }

  outgoing.msg_type = MESSAGE;     // 'M'
  outgoing.id = ID;
  outgoing.descriptor = 0xFF;
  outgoing.payload_length = len;
  outgoing.payload = tx_payload_buf;

  enqueueMsg(&outgoing, len + 6);
  flushTX();
}



void enqueueMsg(MsgStruct *message, uint8_t message_length) {
  (void)message_length; // unused (kept for structure)

  uint8_t last = (uint8_t)(tx_head - 1);

  // if we ended with EOT and want to append more messages, overwrite trailing
EOT
```

```
  if (!tx_buf_cleared && tx_head != tx_tail && tx_buf[last] == EOT) {
    tx_head = last;
  }

  if (tx_buf_cleared) {
    tx_buf[tx_head++] = ENQ;
    tx_buf_cleared = false;
  }

  auto push_tx = [&](uint8_t b) {
    uint8_t next = tx_next(tx_head);
    if (next == tx_tail) {

      return;
    }
    tx_buf[tx_head] = b;
    tx_head = next;
  };

  push_tx(STX);
  push_tx(message->msg_type);
  push_tx(message->id);
  push_tx(message->descriptor);
  push_tx(message->payload_length);

  for (uint8_t i = 0; i < message->payload_length; i++) {
    push_tx(message->payload[i]);
  }

  push_tx(ETX);
  push_tx(EOT);
}

void readSerial() {
  while (Serial.available()) {
    uint8_t incoming_byte = (uint8_t)Serial.read();

    // Minimal overflow guard
    if (rx_head >= RX_BUFSIZE) {
      rx_head = 0;
      rx_tail = 0;
      incoming_message_complete = false;
      return;
    }
```

```
    rx_buf[rx_head++] = incoming_byte;

    if (incoming_byte == EOT) {
      incoming_message_complete = true;
      return;
    }
  }
}

void unpackMessage(MsgStruct *message) {
  message->msg_type = rx_buf[rx_tail++];
  message->id = rx_buf[rx_tail++];
  message->descriptor = rx_buf[rx_tail++];
  message->payload_length = rx_buf[rx_tail++];

  // payload starts here (in rx_buf)
  message->payload_index = rx_tail;

  // advance tail past payload
  rx_tail = (rx_tail + EXPECTED_PAYLOAD_LEN);
}

void flushTX() {
  while (tx_tail != tx_head) {
    Serial.write(tx_buf[tx_tail]);
    tx_tail = (uint8_t)(tx_tail + 1); // uint8_t wrap makes this circular for 256
  }

  tx_buf_cleared = true;
}

void processMessage(MsgStruct *message) {
  uint8_t type = message->msg_type;
  uint8_t descriptor = message->descriptor;
  uint8_t payload_len = message->payload_length;

  if (type != COMMAND) return;

  if (descriptor == LED_DATA_DESCRIPTOR) {
    uint32_t i = 0;
    uint16_t payload_read_index = rx_tail - EXPECTED_PAYLOAD_LEN;
    while((i < NUM_LEDS) && payload_read_index != rx_head ){
      uint8_t r = rx_buf[payload_read_index++];
      uint8_t g = rx_buf[payload_read_index++];
      uint8_t b = rx_buf[payload_read_index++];
```

```
    leds[i] = CRGB(g, r, b);
    i++;
  }
  FastLED.show();
}

if (descriptor == SINGLE_LED_DESCRIPTOR) {
  uint8_t led_index = rx_buf[message->payload_index];
  if (led_index >= NUM_LEDS) return;

  for (uint32_t i = 0; i < NUM_LEDS; i++) {
    leds[i] = CRGB::Black;
  }

  active_led_index = led_index;

  leds[led_index] = CRGB::Red;
  FastLED.show();

  sendASCIIMessage("Turned on single LED:", active_led_index);
}
else if (descriptor == TOGGLE_LED_DESCRIPTOR) {
  uint8_t led_index = rx_buf[message->payload_index];
  if (led_index >= NUM_LEDS) return;

  if (leds[led_index] == CRGB::Black) leds[led_index] = CRGB::Red;
  else if (leds[led_index] == CRGB::Red) leds[led_index] = CRGB::Black;

  FastLED.show();

  sendASCIIMessage("Toggled LED:", led_index);
}
else if (descriptor == ALL_LED_OFF_DESCRIPTOR) {
  for (uint32_t i = 0; i < NUM_LEDS; i++) {
    leds[i] = CRGB::Black;
  }
  FastLED.show();
  sendASCIIMessage("Turned off all LEDs", 0);
}
else if (descriptor == ALL_LED_ON_DESCRIPTOR) {
  for (uint32_t i = 0; i < NUM_LEDS; i++) {
    leds[i] = CRGB::Red;
  }
  FastLED.show();
  sendASCIIMessage("Turned on all LEDs", 0);
```

```
  }

}
```

Flipper Control Code for Expo Demonstration (C++)

```
#include <avr/io.h>
#include <avr/interrupt.h>

#define BUFFER_SIZE 15//define debounce filter length (0.1ms sample rate)

//**********INPUTS***************
// PD0 = Flipper button
// PD1 = EOS Switch

//*********OUTPUTS***************
// PD6 = PWM Output (to MOSFET Gate)
//



void UpdateFlipper0();
void UpdateFlipper1();
void DebounceSwitches();


const uint8_t kHold_Power = 25; // Define Holding OCR0A/B Value
const uint8_t kHit_Power = 255; // Define Flipping 0CR0A/B Value. %DC = 0CR0A / 255

uint8_t flipper_button0 = 0; // Global Flipper state variable
uint8_t EOS_switch0 = 0; //Global EOS State variable

uint8_t flipper_button1 = 0; // Global Flipper state variable
uint8_t EOS_switch1 = 0; //Global EOS State variable


uint8_t switch_states_debounced = 0;  //Global Debounced swtich states



int main(void)
{
```

```
   TCCR0A =
(1<<COM0A1)|(0<<COM0A0)|(1<<COM0B1)|(0<<COM0B0)|(1<<WGM01)|(1<<WGM00); //
Setup Fast PWM Mode for Channel A and Channel B
        TCCR0B = (0<<WGM02)|(0<<CS02)|(1<<CS01)|(0<<CS00); // Define Clock Source
Prescaler
        OCR0A = 0; //Initialized PWM Duty Cycle for flipper 0
        OCR0B = 0; //Initialized PWM Duty Cycle for flipper 1



        //Configure Timer 1
        TCCR1A = (0<<COM1A1)|(1<<COM1A0)|(0<<WGM11)|(0<<WGM10);
        TCCR1B = (0<<WGM13)|(1<<WGM12)|(0<<CS12)|(1<<CS11)|(1<<CS10); //Define clock
source prescaler
        OCR1A = 24; //Set output compare A match every 24+1 clocks (25x40us = 0.1ms)
        TIMSK1 = (1<<OCIE1A); // Enable interrupt for output compare A match

        //Enable Interrupts
        sei();

        DDRD = (1<<PORTD6)| (1<<PORTD5) | (1<<PORTD4); //PWM output D6 (OC0A), PWM
output D5 (OC0B) Debug output D4 (PD0-PD3 Will be inputs)
        DDRB = (1<<PORTB1);  //Timer 1 toggle debug output



   while (1)
   { // Do nothing and wait for Timer 1 to trigger a sample
   }

}

void UpdateFlipper0() {
        static uint8_t flipper_state0 = 0; //Track the state of the flipper
        static uint16_t high_count0 = 0; // Track Duration of high power pulse

        if(flipper_button0){//Button Not Pressed
                flipper_state0 = 0; //Flipper at rest
                OCR0A = 0; //Set at 0 power
                //************************************
        }
        else{//Button Pressed
                switch(flipper_state0) {
                        case 0: //New Flip
                                flipper_state0 = 1;//set state to high power
                                OCR0A = kHit_Power; // Set to flipping power
```

```
                high_count0 = 0; // Reset 40ms pulse counter
                break;
            case 1: //High Power Flip
                if(high_count0<=400){ // Still flipping
                    high_count0++;  //increment to record another 0.1ms
                } else { //Flip over, switch to low power holding
                    flipper_state0=2; //Update state to low power hold
                }
                break;
            case 2: //Low Power Hold
                if(EOS_switch0==0) {  //EOS Switch is closed, flipper is falling
                    flipper_state0=1; //Restart another high powered pulse
                    high_count0=0;
                    OCR0A = kHit_Power; // Hit Power
                }
                else {
                    OCR0A = kHold_Power; // Hold Power
                }
                break;
            default: // state that should never be reached
                flipper_state0 = 0; // Flipper at rest
                OCR0A = 0; // Set power to 0



                }
        }
}

void UpdateFlipper1() {
    static uint8_t flipper_state1 = 0; //Track the state of the flipper
    static uint16_t high_count1 = 0; // Track Duration of high power pulse

    if(flipper_button1){//Button Not Pressed
        flipper_state1 = 0; //Flipper at rest
        OCR0B = 0; //Set at 0 power
        //**********************************
    }
    else{//Button Pressed
        switch(flipper_state1) {
            case 0: //New Flip
                flipper_state1 = 1;//set state to high power
                OCR0B = kHit_Power; // Set to flipping power
                high_count1 = 0; // Reset 40ms pulse counter
```

```
                        break;
                case 1: //High Power Flip
                        if(high_count1<=400){ // Still flipping
                                high_count1++;  //increment to record another 0.1ms
                        } else { //Flip over, switch to low power holding
                                flipper_state1=2; //Update state to low power hold
                        }
                        break;
                case 2: //Low Power Hold
                        if(EOS_switch1==0) {  //EOS Switch is closed, flipper is falling
                                flipper_state1=1; //Restart another high powered pulse
                                high_count1=0;
                                OCR0B = kHit_Power; // Hit Power
                        }
                        else {
                                OCR0B = kHold_Power; // Hold Power
                        }
                        break;
                default: // state that should never be reached
                        flipper_state1 = 0; // Flipper at rest
                        OCR0B = 0; // Set power to 0



                        }
        }
}

void DebounceSwitches() {
        static uint8_t switch_states_buffer[BUFFER_SIZE]; //Stores previous and current switch
states
        static uint16_t i_buffer = 0; // index to track where we are in buffer
        uint8_t rising_edge_detect = 0xFF; // Initialize rising edge detect to rising
        uint8_t falling_edge_detect = 0; // Initialize falling edge detect to falling

        switch_states_buffer[i_buffer] = PIND&0x0F; // Record current switch states

        //Debounce Swtich States
        // Loop through all previous switchs states
        for(uint16_t i=0; i<(BUFFER_SIZE-1); i++) {
                // If i-buffer = Buffer_SIZE - 1 handle wrap around
                if(i_buffer==(BUFFER_SIZE-1)) {
                        //i==0 is the oldest sample (wrap around)
                        if(i==0){
```

```
                        rising_edge_detect &= ~switch_states_buffer[i];
                        falling_edge_detect |= ~switch_states_buffer[i];
                        }
                else {
                        rising_edge_detect &= switch_states_buffer[i];
                        falling_edge_detect |= switch_states_buffer[i];
                        }
                }
        //No wrap around
        else {
                //i==1 buffer+1 is the oldest sample
                if(i==i_buffer+1){
                        rising_edge_detect &= ~switch_states_buffer[i];
                        falling_edge_detect |= ~switch_states_buffer[i];
                }
                else {
                        rising_edge_detect &= switch_states_buffer[i];
                        falling_edge_detect |= switch_states_buffer[i];
                }
                }
        }
        //Updated switch debounce states based on edge detections
        switch_states_debounced &= falling_edge_detect;
        switch_states_debounced |= rising_edge_detect;

        flipper_button0 = switch_states_debounced&(1<<PIND0);
        EOS_switch0 = switch_states_debounced&(1<<PIND1);

        //THIS HAS
CHANGED**********************************************888888888888
        flipper_button1 = switch_states_debounced&(1<<PIND2);
        EOS_switch1 = switch_states_debounced&(1<<PIND3);

        //Debug output if we need to check EOS State
        //if(EOS_switch) {
        //  PORTD |= (1<<PORTD2);}
        //else{
        //        PORTD &= ~(1<<PORTD2);}

        //Update buffer index to be ready to store next switch reading
        //Need to handle wrap around condition
        if(i_buffer==(BUFFER_SIZE-1)) {
                i_buffer=0;
        } else {
```

```
                        i_buffer++;
                }
        }



ISR(TIMER1_COMPA_vect) {
        DebounceSwitches();
        UpdateFlipper0(); //Update Flipper State
        UpdateFlipper1(); //Update Flipper 1 state
}
```